

CHAPTER 1

INTRODUCTION

The Enterprise Hospital Management System (HMS) is a comprehensive digital solution designed to automate and manage the day-to-day operations of a hospital or healthcare organization. Many processes, like patient registration, appointment scheduling, billing, and record management, are done manually or through disconnected systems in traditional hospitals. This frequently results in errors, delays, data duplication, inefficiencies, and delays. A centralized and integrated system that can streamline operations and enhance the quality of healthcare services is required to overcome these obstacles. Modern web technologies like React.js for the front end, Node.js for the back end, and MySQL for database management are used to create the proposed Hospital Management System. Administrators, doctors, and patients all have access to a single platform through which they can communicate effectively. It enables hospitals to store lab reports in a structured and secure manner, manage appointments, generate bills, manage pharmacy inventory, and keep accurate records. This system's primary goals are to reduce manual labor and increase operational efficiency. By digitizing hospital processes, the system minimizes paperwork, reduces human errors, and speeds up data processing. Additionally, it ensures that patient data is kept safely and can be accessed quickly whenever required. The system supports role-based access control, ensuring that only authorized users can access specific functionalities and sensitive data.

The system's capacity to improve the experience of patients is yet another important feature. Through the platform, patients can book appointments, access their medical reports, and receive billing information without waiting around. Doctors can efficiently manage their schedules and access patient histories, enabling better diagnosis and treatment planning. Analytics and dashboards let

administrators keep an eye on what's going on in the hospital, allowing them to make better decisions. The system is designed to be scalable, flexible, and user-friendly, making it suitable for small clinics as well as large hospitals. The Enterprise Hospital Management System is a one-stop shop for today's healthcare requirements thanks to its integrated modules for pharmacy and laboratory management, real-time data access, and automated billing. In conclusion, the Hospital Management System aims to transform traditional healthcare management into a smart, efficient, and digital ecosystem, improving both operational performance and patient care quality.

Frontend:

The frontend of the Enterprise Hospital Management System is in charge of making the system's interactive and simple-to-use interface available to administrators, doctors, and patients. It serves as the application's presentation layer, facilitating user-to-backend service communication. Modern web technologies like React.js, HTML, CSS, JavaScript, and Tailwind CSS are used to create the frontend, resulting in a user experience that is responsive, dynamic, and visually appealing. React.js's component-based architecture is used to create the system's frontend. Forms, dashboards, navigation bars, and tables are just a few examples of reusable UI components that can be created using this strategy. The system is easier to maintain and scale because each component handles a specific function. For example, components are used for patient registration forms, appointment booking interfaces, billing pages, and report viewing modules. This modular design improves code organization and enhances the overall performance of the application.

The frontend's responsive design is one of its most important features because it makes the system accessible from desktops, tablets, and smartphones. Tailwind CSS is used to create interfaces that are sleek and contemporary, have adaptable layouts, consistent styling, and render quickly. The user interface is structured in a way that allows users to navigate easily between different modules such as appointments, billing, pharmacy, and lab reports.

Role-based dashboards are another feature of the frontend that lets users choose which interfaces they see based on their roles. For example, administrators can access system analytics, manage staff, and monitor operations, while doctors can view patient records and manage appointments. On the other hand, patients are able to download reports, view prescriptions, and schedule appointments. This separation of interfaces improves usability and ensures that users only access relevant features.

The frontend's real-time interaction with the backend via RESTful APIs is another important feature. The frontend sends a request to the backend server whenever a user completes an action, such as creating a bill or booking an appointment, and displays the response dynamically without reloading the page. This results in a user experience that is both smooth and effective. Events are handled, forms are validated, and the interface is updated dynamically using JavaScript. The frontend also includes data visualization features using libraries such as Chart.js. These visual elements are used in dashboards to display important information like patient statistics, revenue trends, and appointment summaries. Graphical representations help administrators quickly understand system data and make informed decisions.

Security is also considered at the frontend level by implementing form validation, input sanitization, and secure authentication handling. In order to reduce errors and increase system reliability, proper validation ensures that only correct data is sent to the backend. In conclusion, the frontend of the Enterprise Hospital Management System plays a crucial role in delivering a smooth, responsive, and efficient user experience. It provides a user-friendly interface that simplifies hospital operations and enhances user interaction with the system by utilizing cutting-edge technologies and design principles.

Backend:

The backend of the Enterprise Hospital Management System plays a vital role in managing the core functionality, processing data, and ensuring smooth communication between the frontend and the database. It acts as the application layer where all business logic is implemented and system operations are carried out efficiently. The backend is developed using Node.js along with the Express.js framework, while MySQL Workbench is used for designing and managing the database.

Node.js provides a powerful server-side runtime environment that allows JavaScript to be executed outside the browser. It follows an asynchronous and non-blocking architecture, which enables the system to handle multiple requests simultaneously without affecting performance. This feature is highly beneficial in a hospital environment where many users such as administrators, doctors, and patients access the system at the same time. Express.js is used as a backend framework to simplify the development process by providing features such as routing, middleware support, and easy API creation.

The backend is responsible for handling various operations such as patient registration, appointment scheduling, billing management, pharmacy updates, and laboratory report processing. When a user performs an action on the frontend, such as booking an appointment or generating a bill, the request is sent to the backend through RESTful APIs. The backend processes the request, performs the required logic, interacts with the database, and sends the appropriate response back to the frontend. This ensures real-time data processing and improves the overall user experience.

MySQL Workbench is used as the database management tool for designing and maintaining the system database. It provides a graphical interface to create tables, define relationships, and execute SQL queries efficiently. The database stores all important information such as patient details, doctor records, appointments, billing data, pharmacy inventory, and lab reports. Relationships between tables are established using primary and foreign keys, which help maintain data consistency and integrity across the system.

Security is an important aspect of the backend. The system implements authentication and authorization mechanisms to ensure that only authorized users can access specific functionalities. Techniques such as JSON Web Tokens (JWT) or session-based authentication are used to verify user identity. Input validation and data sanitization are also performed to prevent common security threats such as SQL injection and unauthorized data access. Secure API communication further enhances the protection of sensitive patient information.

The backend is designed using a modular architecture where different functionalities are divided into separate modules such as patient management, appointment management, billing, pharmacy, and laboratory systems. Each module performs a specific task but works together as part of the overall system. This modular design improves code organization, makes maintenance easier, and allows new features to be added without affecting existing functionalities.

In addition, the backend supports efficient error handling and logging mechanisms to ensure system reliability. It can detect and handle issues such as invalid inputs, server errors, and database failures without interrupting the system's operation. This helps maintain system stability and ensures continuous service availability.

In conclusion, the backend of the Enterprise Hospital Management System is essential for handling system operations, managing data, and ensuring secure communication. By using Node.js, Express.js, and MySQL Workbench, the system achieves high performance, scalability, and reliability, making it suitable for modern healthcare management.

1.1 Project Overview

The Enterprise Hospital Management System (HMS) is a web-based application designed to automate and manage the overall operations of a hospital in an efficient and organized manner. In traditional healthcare systems, many processes such as patient registration, appointment scheduling, billing, pharmacy management, and laboratory report handling are performed manually or through disconnected systems. This often leads to inefficiencies, data duplication, delays, and increased chances of human error.

To overcome these issues, the proposed system provides a centralized digital platform that integrates all hospital departments into a single system. The application is developed using modern technologies such as React.js for the frontend, Node.js for the backend, and MySQL for database management. This combination ensures a responsive user interface, fast data processing, and secure data storage.

The system is designed to be used by three main types of users: administrators, doctors, and patients. Administrators can manage staff, monitor operations, and view analytical reports. Doctors can access patient records, manage appointments, and update treatment details. Patients can book appointments, view prescriptions, and access their medical reports online.

The HMS improves overall efficiency by automating routine tasks, reducing paperwork, and enabling real-time data access. It also enhances patient experience by reducing waiting time and providing easy access to healthcare services. The system is scalable, flexible, and suitable for both small clinics and large hospitals.

1.2 Problem Statement

By providing individuals with essential medical services, the healthcare industry plays a crucial role in society. However, many healthcare facilities still use semi-manual and traditional management systems to run their day-to-day operations. Inefficiencies, delays, and an increased risk of human error are all consequences of these out-of-date systems, which frequently involve manual record-keeping procedures, disconnected software tools, and paperwork. Traditional methods of managing hospital operations become increasingly difficult and ineffective as the number of patients and healthcare services increases. One of the major problems in existing hospital systems is the lack of centralized data management. Medical reports, appointment details, and patient records are frequently kept in separate systems or physical files. Because of this fragmentation, it is challenging for healthcare professionals to quickly gain access to comprehensive patient data. For instance, a patient's medical history or lab results may not be immediately accessible to a doctor, which can delay diagnosis and treatment. Such delays can have a significant impact on patient outcomes in emergency situations. Another significant issue is the inefficient appointment scheduling process.

Appointments are typically booked either manually or through basic systems that do not provide real-time doctor availability. Conflicts in scheduling, lengthy wait times, and a poor patient experience may result from this. To book or reschedule appointments, patients may need to physically visit the hospital or call multiple times, which is time-consuming and inconvenient. Traditional billing procedures also come with a number of difficulties.

There is a greater chance of calculation errors, duplicate entries, and invoice generation delays when billing manually. Accurate financial records may be difficult to keep for hospitals, which can have an impact on revenue management and auditing procedures. Additionally, patients may experience confusion or dissatisfaction due to unclear or incorrect billing information. Another problem is the lack of efficient pharmacy and inventory management. A consistent supply of medications and medical supplies is essential for hospitals. It becomes difficult to keep track of stock levels, expiration dates, and usage patterns without a proper inventory management system. This can result in problems like overstocking, stock shortages, or the use of expired medications, which can have a negative effect on patient safety and hospital operations. The management of laboratory reports and medical documents is also a major concern. Lab reports are frequently stored as scattered digital files or physical documents in conventional systems. Retrieving these reports can be time-consuming and may result in misplacement or loss of important medical information. Patients may also face difficulty accessing their reports when needed.

Another critical issue is the lack of role-based access and data security. To maintain patient confidentiality and privacy, sensitive information must be safeguarded. In many existing systems, there is limited control over who can access or modify data. Unauthorized access, data breaches, and the misuse of confidential information are all more likely as a result of this. In healthcare systems, it is essential to ensure that data access is controlled and secure. Furthermore, traditional hospital systems often lack real-time data analytics and reporting capabilities. Hospital administrators need accurate and up-to-date information to make informed decisions regarding patient care, resource allocation, and financial management. Without proper analytics tools, it becomes difficult to monitor hospital performance, identify trends, and improve operational efficiency.

Another major issue is the inability of various hospital departments to work together. Reception, pharmacy, laboratory, and billing are just a few of the departments that frequently operate on their own without proper coordination. This leads to communication gaps, duplication of work, and delays in service delivery. To ensure that all departments are able to communicate and work together effectively, an integrated system is required. Additionally, the increasing demand for healthcare services requires systems that are scalable and capable of handling large volumes of data and users. During peak times, slow performance and system failures can occur as a result of traditional systems' inability to efficiently handle high workloads. A comprehensive and integrated Hospital Management System that can automate and streamline hospital operations is required to address these issues. The goal of the proposed system is to provide a centralized platform from which all hospital operations can be effectively managed. By leveraging modern technologies such as web-based applications, real-time databases, and secure APIs, the system can improve data accessibility, reduce manual effort, enhance patient experience, and ensure better decision-making.

In conclusion, the current approaches to managing hospitals face a number of issues, including ineffective data management, scheduling issues, billing errors, inadequate inventory control, lack of security, and the absence of real-time analytics. These difficulties emphasize the necessity of a cutting-edge, integrated, automated solution that can guarantee high-quality patient care while simultaneously increasing the overall efficiency and effectiveness of hospital operations.

Drawbacks:

Although the Enterprise Hospital Management System (HMS) offers significant advantages over conventional methods, there are some drawbacks and difficulties that must be taken into account. It is essential to comprehend these drawbacks if the system is to be improved and its effective implementation in real-world healthcare settings is to be ensured. One of the primary drawbacks is the high initial development and implementation cost. Developing a full-scale hospital management system requires investment in software development, hardware infrastructure, and database setup. Servers, networking hardware, and

security systems may also be required to be purchased by hospitals. For small clinics or healthcare centers with limited budgets, these costs can be a major barrier.

The requirement for technical knowledge and training is another limitation. Digital systems may be unfamiliar to administrators, doctors, and nurses at the hospital. They require proper training to use the software effectively. Users may encounter difficulties operating the system if adequate training is not provided, which may result in errors and a decrease in productivity. In addition, the system is heavily dependent on system availability and connectivity to the internet. A reliable internet connection is required for data access and operation because the application is web-based. Network failures, server downtime, or technical issues can disrupt hospital activities and delay access to critical information.

Risks to privacy and data security are another issue. Personal information, billing information, and sensitive patient data are all stored in the system. Data breaches, unauthorized access, and cyberattacks are all possible outcomes of inadequate security measures. Ensuring strong encryption, authentication, and access control is essential to minimize these risks.

The maintenance and update requirements of the system can also be challenging. Regular updates are needed to fix bugs, improve performance, and add new features. This requires continuous technical support and system monitoring. The system may become out of date or vulnerable to security threats if it is not properly maintained. The possibility of software bugs or system errors is another limitation. Even well-designed systems may encounter technical issues such as incorrect data processing, system crashes, or unexpected behavior. These issues can affect hospital operations and require immediate attention from technical teams.

If the system is not properly designed, it may also face difficulties with scalability. The system must be capable of handling increased workloads effectively as the number of users and data increases. If scalability is not considered, the system may experience slow performance or delays during peak usage.

The staff's resistance to change is another disadvantage. Some users may find it challenging to move from manual systems to digital platforms. Employees may be reluctant to adopt new technology due to lack of familiarity or fear of change. The system's successful implementation may be affected by this. Additionally, the system may only offer limited customization options. Different hospitals may have unique workflows and requirements. Without additional customization, a standard system may not fully satisfy all specific requirements, which can increase development time and cost.

Finally, the system's dependability and backup mechanisms are essential. Backup and recovery systems must be in place in the event of data loss, server failure, or technical issues. Important hospital data could be lost without dependable backup solutions, affecting patient care and administrative processes.

1.3 Objective

By making use of cutting-edge technology, the Enterprise Hospital Management System (HMS) aims to enhance the overall quality, accuracy, and efficiency of hospital operations. The system aims to replace traditional manual processes with an integrated digital platform that simplifies hospital management and enhances patient care. Automation, data management, user accessibility, security, and support for decision-making are the main goals of this system. The system's primary objective is to automate hospital operations. In traditional systems, many tasks such as patient registration, appointment scheduling, billing, and record management are handled manually. This frequently results in inefficiencies, errors, and delays. The proposed system aims to automate these processes, which will speed up and improve operations and reduce the amount of manual labor required. Automation helps hospital staff focus more on patient care rather than administrative tasks.

The provision of centralized data management is yet another significant objective. A single, integrated database is intended to house all hospital-related data, including patient records, appointments, billing information, pharmacy details, and lab reports. Data access is made simple and quick by this centralized strategy for authorized users. Additionally, it improves consistency across various hospital departments and eliminates duplicate data.

Additionally, the system aims to improve patient satisfaction and experience. The system makes it easier for patients to interact with the hospital by offering features like easy access to medical records, online appointment booking, and digital billing. Patients can save time, avoid long waiting periods, and receive faster services. Better patient trust and contentment with healthcare services result from an enhanced patient experience. Getting better at managing appointments and schedules is another important goal. The system enables doctors to effectively manage their schedules and lets patients book appointments online. It prevents scheduling conflicts and makes doctors available in real time. As a result waiting times are cut down and hospital resources are better utilized. Additionally, the system aims to guarantee transparent and accurate billing. Charges are determined by the services provided, medications, and laboratory tests by the automated billing module. It generates invoices quickly and reduces the chances of errors.

Transparent billing improves trust between patients and the hospital and ensures proper financial management. Efficient pharmacy and inventory management is another important objective of the system. The system keeps track of medicine stock levels, expiration dates, and real-time inventory updates. This ensures that medicines are available when needed, reduces waste, and helps prevent shortages. For a hospital to run smoothly, it is essential to manage its inventory properly. Additionally, the system is intended to enhance data privacy and security. Protecting sensitive patient information is essential. Data encryption, role-based access control, and authentication mechanisms are all incorporated into the system to guarantee that only authorized users have access to specific data. This helps maintain confidentiality and prevents unauthorized access to critical information.

Real-time analytics and reporting are another objective. The system has dashboard features that show important data like appointment trends, revenue reports, and patient statistics. These insights help hospital administrators make informed decisions, improve resource allocation, and enhance overall performance.

Additionally, the system aims to unite various hospital departments on a single platform. Departments such as reception, pharmacy, laboratory, and billing are connected through the system, ensuring smooth communication and coordination. This integration reduces duplication of work and improves workflow efficiency.

The system's scalability is another important goal. As the hospital expands, the system is built to handle an increasing number of users and data. This guarantees that the system will continue to function effectively and reliably despite heavy workloads. The system is suitable for both small clinics and large hospitals due to its scalability. Accessibility and user-friendliness are also major concerns for the system. The user interface is designed to be simple and easy to use, making it possible for users with varying levels of technical expertise to effectively use the system. The responsive design ensures that the system can be accessed from various devices such as computers, tablets, and smartphones.

Another objective is to reduce paperwork and promote a digital environment. By storing data electronically and automating processes, the system reduces the need for physical documents. This not only contributes to environmentally friendly practices but also saves space and time. Lastly, the system aims to increase healthcare efficiency and quality overall. The system helps healthcare professionals deliver high-quality medical services by providing accurate data, speeding up processing, and improving coordination. It contributes to the reduction of errors, enhancement of patient care, and enhancement of the hospital's overall performance.

1.4 Scope of Study

The proposed Enterprise Hospital Management System is a fully integrated and automated solution designed to address the limitations of the existing system. It provides a centralized platform where all hospital activities are managed efficiently with improved accuracy and speed.

One of the key features of the proposed system is centralized data management. All information such as patient details, appointments, billing records, pharmacy inventory, and lab reports are stored in a single database. This ensures quick access to data and eliminates redundancy. Doctors can easily retrieve patient history, which helps in better diagnosis and treatment.

The system also includes an automated appointment scheduling module that allows patients to book, reschedule, or cancel appointments online. Real-time doctor availability helps in avoiding scheduling conflicts and reduces waiting time. In addition, the billing module automatically calculates charges and generates accurate invoices, improving transparency and reducing manual errors.

Pharmacy and inventory management is another important feature, where stock levels, expiry dates, and medicine usage are tracked in real time. The laboratory module enables secure uploading and easy access to digital reports, eliminating the need for physical documents.

Security is ensured through role-based access control and authentication mechanisms, allowing only authorized users to access sensitive data. The system also provides dashboards and analytics to help administrators monitor performance and make informed decisions. Overall, the proposed system enhances efficiency, improves data management, and provides better healthcare services.

CHAPTER 2

SYSTEM ARCHITECTURE

2.1 Existing System

In many healthcare facilities, the current hospital management system is primarily based on manual, semi-digital, or traditional procedures. Patients' registration, appointment scheduling, billing, pharmacy management, and laboratory reporting are just a few of the basic hospital operations that these systems are designed to handle. However, these conventional systems are no longer sufficient to effectively manage complex hospital workflows due to the rapid expansion of the healthcare industry and the rising demands of patients. Numerous operational difficulties have resulted from the absence of integration, automation, and real-time data access. In the current system, patient information is often maintained using paper-based records or simple standalone software applications. When a patient visits the hospital, their details are recorded manually at the reception desk. After that, these records are kept in basic computer systems or in physical files. Since these records are not centralized, accessing patient information becomes time-consuming. Diagnosis and treatment can be slowed down by doctors having to search through files or request records from other departments. The appointment scheduling process in the existing system is also inefficient. Patients typically book appointments by visiting the hospital in person or by making phone calls. This manual process does not provide real-time information about doctor availability, which can lead to scheduling conflicts or overbooking. As a result, patients frequently have to wait a long time, and doctors may have trouble effectively managing their schedules. Another major issue in the existing system is the lack of efficient communication between hospital departments. The reception, pharmacy, laboratory, and billing departments frequently operate independently. The lack of coordination between these departments causes service delivery delays. For instance, a doctor might order a test, but the laboratory might not get the information right away, which would delay the production of the report. Similarly, billing departments may not have real-time access to services provided, leading to inaccurate or delayed billing.

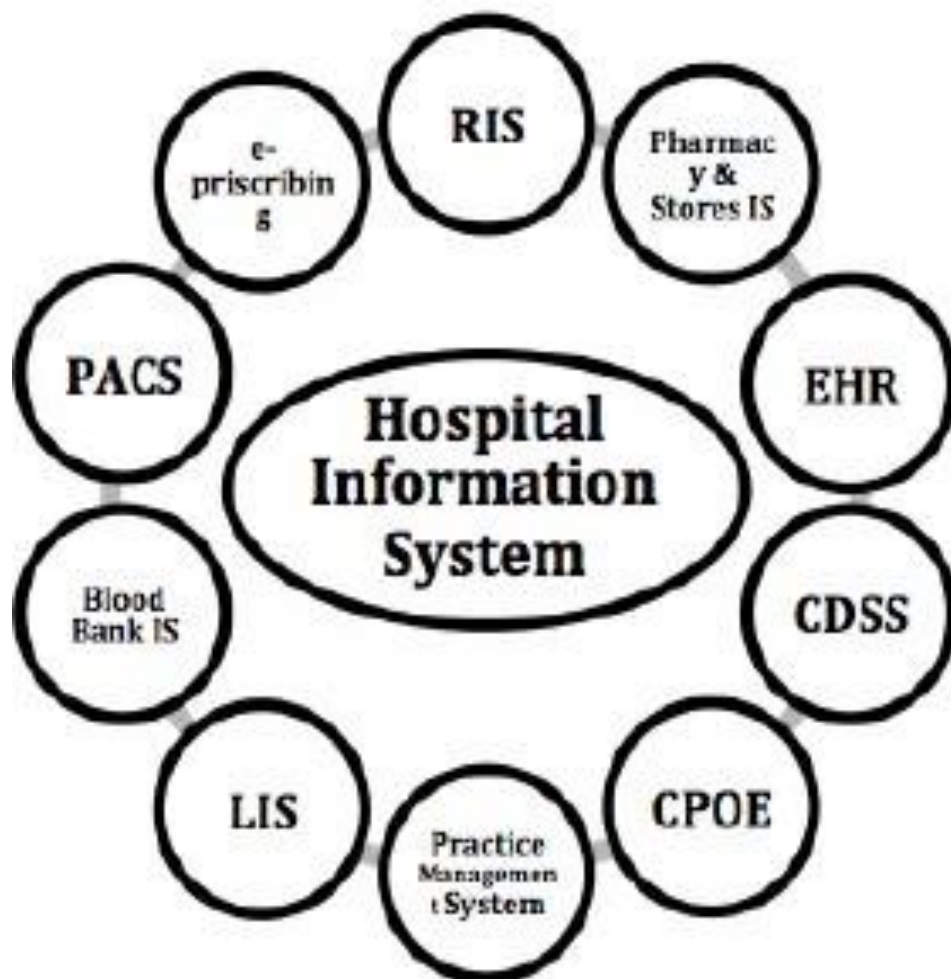
The billing process in traditional systems is mostly handled manually. Invoices are generated by staff members after charges for consultations, treatments, medications, and lab tests are calculated. Human errors like incorrect calculations, entries that are missing, or duplicate records are common in this procedure. Additionally, manual billing takes longer, which can lead to patient inconvenience and long lines. Additionally, the existing pharmacy and inventory management system is limited. The tracking of medicines is done either manually or through basic systems that do not provide updates in real time. This makes it difficult to monitor stock levels, track expiry dates, and manage supply efficiently. As a consequence of this, operational efficiency and patient safety can be negatively impacted by issues like stock shortages, overstocking, or the utilization of expired medications in hospitals.

The management of laboratory reports and medical records is another challenge. Without proper organization, lab reports are frequently stored in physical files or digital documents. When patients require these reports immediately, retrieving them can take a long time. Important medical information may also be misplaced or lost, which could have an impact on patient care. The existing system's lack of data security and access control is yet another significant drawback. Sensitive patient information is often not protected with proper security measures. Unauthorized access and data breaches are more likely if there are no clear restrictions on who can access or modify data. Maintaining confidentiality and privacy of patient data is a critical requirement in healthcare, which is not adequately addressed in traditional systems.

Analytics and reporting capabilities in real time are also lacking in the current system. Hospital administrators need accurate data to monitor performance, manage resources, and make strategic decisions. However, in traditional systems, generating reports requires manual effort and time. It is difficult to monitor patient trends, revenue, and operational effectiveness without real-time analytics. Another limitation is the lack of scalability and flexibility.

A growing number of patients or large amounts of data cannot be handled by conventional systems. As hospitals grow, these systems become slow and inefficient, making it difficult to manage operations effectively. Additionally, it may be challenging to update or modify these systems to meet new requirements. Additionally, the current system relies heavily on manual procedures and paperwork, which adds to the workload of hospital employees. Managing physical records requires storage space and proper organization, and there is always a risk of data loss due to damage or misplacement. Additionally, manual procedures increase the likelihood of errors and reduce productivity.

2.1.1 Existing System Architecture:



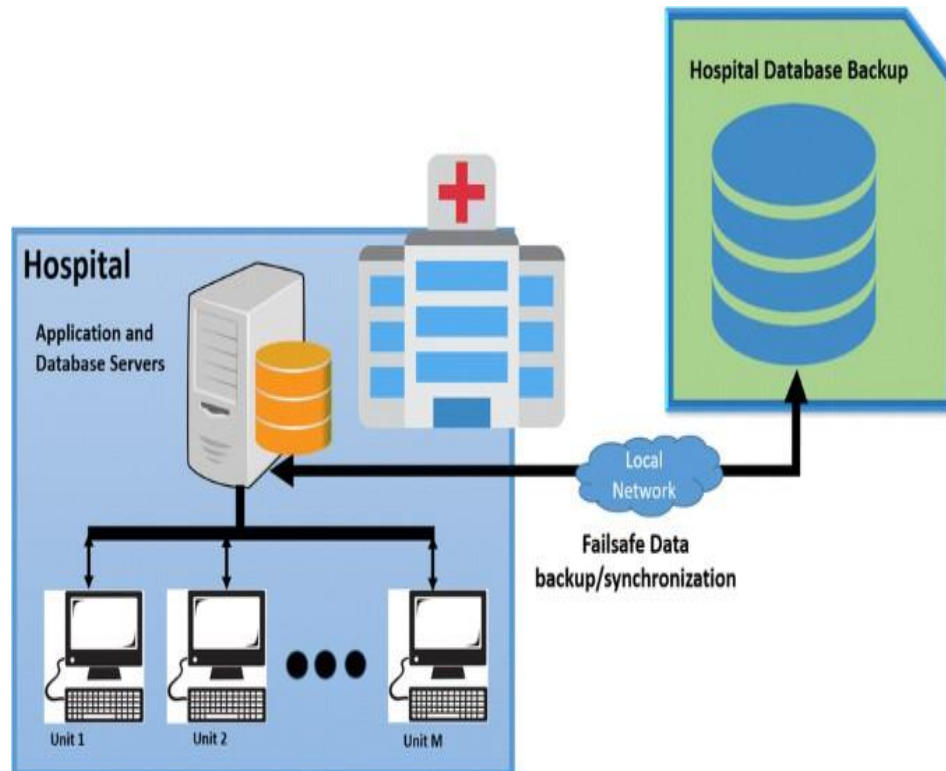


Fig:2.1.1.1 Existing System Architecture

Explanation:

The existing system architecture in many healthcare facilities is characterized by manual, semi-digital, and fragmented procedures that are increasingly inadequate for modern healthcare demands.

Core Characteristics of the Existing Architecture

The current architecture is largely defined by a lack of integration and automation across various hospital departments. It typically operates through:

- **Manual Record-Keeping:** Patient information is frequently maintained using paper-based records or simple, standalone software applications. When a patient arrives, their details are manually recorded at the reception desk and subsequently stored in physical files or basic, non-centralized computer systems.
- **Disconnected Departmental Modules:** Departments such as reception, pharmacy, laboratory, and billing often operate independently. This "siloe" structure means that information does not flow automatically between units; for instance, a laboratory may not receive a doctor's test order immediately, leading to delays in diagnosis and treatment.

- **Manual Appointment and Billing Processes:** Appointment scheduling is typically handled via in-person visits or phone calls without real-time visibility into doctor availability. Similarly, billing is a manual calculation process where staff members must aggregate charges for consultations, medications, and tests, making it highly susceptible to human errors like duplicate records or incorrect calculations.

Operational Challenges and Drawbacks

The architectural reliance on manual effort and fragmented tools leads to several significant operational hurdles:

- **Inefficient Data Access:** Because records are not centralized, healthcare professionals must often search through physical files or request data from other departments, which is time-consuming and can be dangerous in emergency situations.
- **Poor Inventory Management:** Pharmacy and inventory tracking are often handled through basic systems that lack real-time updates. This results in stock shortages, overstocking, or the potential use of expired medications, directly impacting patient safety.
- **Security and Scalability Limitations:** Traditional systems frequently lack robust data security measures and role-based access controls, increasing the risk of unauthorized data breaches. Furthermore, these systems are not scalable; as patient volume grows, the manual and semi-digital infrastructure becomes slow and inefficient.
- **Lack of Analytics:** There is a notable absence of real-time reporting capabilities. Administrators must exert significant manual effort to generate reports, making it difficult to monitor hospital performance, revenue trends, or operational effectiveness.

2.1.1 Disadvantages

There are a number of drawbacks with the current hospital management system that have an impact on the effectiveness, accuracy, and overall performance of healthcare services. It is largely based on manual processes or fragmented digital tools. Hospital staff face operational difficulties as a result of these restrictions, which also lower the standard of patient care and increase administrative workload. Understanding these disadvantages is essential for identifying the need for a modern, integrated hospital management system.

The absence of centralized data management is one of the main drawbacks of the current system. In traditional systems, patient information, medical records, billing details, and laboratory reports are stored in different locations, either as physical documents or in separate software systems. Because of this fragmentation, finding complete and accurate information when needed is difficult. Frequently, doctors and staff spend more time looking for records, delaying diagnosis and treatment. The inability to quickly retrieve patient information can have serious consequences in emergency situations. The manual procedures, which are time-consuming and inefficient, are yet another significant drawback. Most of the time, manual processes are used to handle things like patient registration, appointment scheduling, billing, and managing reports. The workload of hospital staff is increased, and productivity is decreased as a result. Errors like duplicate records, misplaced documents, and incorrect data entry are also more likely to occur in manual processes. Both patient care and administrative operations may be impacted by these errors. In traditional setups, the appointment scheduling system is also inefficient. Usually, appointments are handled over the phone or in person, and there are no real-time updates on the doctor's availability. Overbooking, patient wait times, and scheduling conflicts are all possible outcomes of this. Poor appointment management not only affects patient satisfaction but also reduces the efficiency of doctors' schedules.

The lack of integration between various hospital departments is yet another major drawback. Reception, the pharmacy, the lab, and billing are all departments that frequently operate on their own, with little coordination. As a result of this lack of integration, there are gaps in communication, work that is done twice, and service delivery delays. The entire treatment process can be slowed down, for instance, if information is not transferred promptly from the consultation department to the laboratory. In addition, the billing procedure in the current systems is prone to errors and inefficiency. Manual billing increases the chances of incorrect calculations, missing entries, and delayed invoice generation. Patients may become confused as a result, and the hospital's financial management may be impacted. Additionally, it is challenging to efficiently conduct audits and keep accurate financial records due to the absence of automated billing systems. Poor inventory management is another disadvantage of pharmacy operations. It is difficult to keep track of

medicine stock levels, expiration dates, and usage patterns without an automated system. As a result, essential medicines may be out of stock or items that are likely to expire may be overstocked. Hospital operations are impacted by improper inventory management, which can also compromise patient safety. Another obstacle is managing medical records and laboratory reports. Reports are frequently stored as unorganized digital documents or physical files in conventional systems. There is a possibility of misplacing or losing important information during the retrieval of these reports, which can take a long time. The continuity of patient care may be compromised as a result of this inadequate record management. Another critical issue with the current system is data security. The absence of proper security measures and access control mechanisms increases the risk of unauthorized access to sensitive patient information. The lack of strict controls over who can view or modify data frequently results in potential privacy breaches. It is essential to protect patient data, and traditional systems frequently fail to meet current security standards.

The absence of real-time analytics and reporting is another drawback. To monitor operations, manage resources, and make strategic decisions, hospital administrators require accurate and up-to-date data. In traditional systems, generating reports requires manual effort and is often time-consuming. Trends can be difficult to identify, processes can be improved, and overall performance can be improved without real-time insights. Scalability issues are also present in the existing system. As the number of patients and hospital services increases, traditional systems struggle to handle large volumes of data efficiently. Slow performance, system delays, and difficulty managing operations are all possible outcomes of this. Healthcare facilities have limited growth potential due to a lack of scalability. The excessive reliance on paper work and physical storage is another drawback. Managing paper-based records requires significant storage space and proper organization. Data loss from file damage, theft, or misplacement is always a possibility. Additionally, physical record keeping wastes time and is bad for the environment. Lastly, the current system is inefficient and lowers patient satisfaction. Delays in service delivery, long waiting times, and errors in data handling negatively impact the patient experience. Patients expect quick and effective healthcare services, which traditional systems fail to deliver in today's fast-paced world.

2.2 Proposed System

The enterprise hospital management system (HMS) that is being proposed is a complete, web-based solution that is made to automate and integrate all of the major hospital operations into a single, centralized platform. Through improved patient care, secure access, efficient data handling, and real-time communication, the system aims to overcome the limitations of traditional hospital management. The proposed system ensures high performance, scalability, and dependability by utilizing cutting-edge technologies like MySQL for database management, React.js for the frontend, Node.js for the backend, and React.js for the frontend. The proposed system's centralized data management is a key feature. A single database houses all information pertaining to the hospital, such as patient records, appointments, billing information, pharmacy inventory, and laboratory reports. This allows authorized users to access accurate and up-to-date information instantly. Patients' histories, lab results, and prescriptions can be easily retrieved by doctors, allowing for quicker and better-informed decisions. Data duplication is also reduced and uniformity is maintained across all departments with centralized storage. An automated appointment management module that lets patients book, change their appointment times, or cancel them online is added to the system. As a result of the system's real-time availability of doctors, scheduling conflicts are avoided and time management is improved. This feature reduces waiting times and enhances the overall patient experience.

Additionally, doctors can effectively manage their schedules, resulting in better utilization of hospital resources. The proposed system's automated billing and invoice generation module is another important feature. The system instantly generates accurate invoices and calculates costs based on consultations, treatments, medications, and lab tests. It also supports PDF invoice generation, allowing patients to download and store their billing information. This automation speeds up the billing process, increases transparency, and reduces manual errors. A pharmacy and inventory management module is included in the proposed system, allowing for real-time monitoring of medicine stock levels, expiration dates,

and usage patterns. The system automatically updates inventory after each transaction and provides alerts for low stock or expired medicines. This guarantees effective inventory management, decreases waste, and enhances patient safety. Digital lab reports can be securely uploaded and stored with the help of the laboratory management module. The system makes it simple for patients and doctors to access these reports, removing the need for paper copies. This improves record management and ensures that important medical information is always available when needed.

Security is a major focus of the proposed system. It implements role-based access control (RBAC), ensuring that users such as administrators, doctors, and patients can only access the features and data relevant to their roles. Authentication mechanisms such as secure login and encrypted data storage protect sensitive patient information. This ensures privacy, confidentiality, and compliance with healthcare data standards.

The system also provides interactive dashboards and analytics for administrators. Key performance indicators like patient statistics, revenue reports, appointment trends, and resource utilization are shown on these dashboards. The system presents data in graphical formats using tools like Chart.js, making it easier for administrators to analyze and make informed decisions. Another important aspect of the proposed system is integration between different hospital departments. All departments, including reception, pharmacy, laboratory, and billing, are connected through a single platform.

As a result, there are fewer delays and the efficiency of the workflow is increased because of this seamless communication and coordination. For instance, when a physician orders a test, the laboratory immediately receives the request, and the billing department receives an automatic update. As the hospital expands, the proposed system will be able to handle an increasing number of users and data thanks to its scalability and adaptability. The modular design makes it simple to update or add new features without affecting the entire system. Long-term usability and adaptability to shifting healthcare requirements are guaranteed by this. Accessibility and user-friendliness are also major concerns for the system.

Users can access the system from desktops, tablets, and smartphones thanks to the system's responsive, easy-to-use interface. This makes the system easier to use and ensures that users of all technical abilities can easily interact with it. Additionally, the system encourages a completely digital environment, which reduces paperwork dependence. Physical storage is no longer required for electronic records, data loss is less likely to occur, and environmental sustainability is enhanced.

2.1.3 Features

The Enterprise Hospital Management System (HMS) is a platform that automates and streamlines various hospital operations. It is designed to be comprehensive and integrated. The system has a number of features that work together to make healthcare services more effective, accurate, and of higher quality. Modern technologies were used to create these features, which are organized to meet the requirements of administrators, doctors, and patients in a unified environment. One of the primary features of the system is Role-Based Access Control (RBAC).

This feature ensures that different users have access to specific functionalities based on their roles within the hospital. Doctors, on the other hand, have access to patient records, can schedule appointments, and upload prescriptions. Administrators, on the other hand, have complete control over system operations like managing staff, departments, and reports. On the other hand, patients are able to schedule appointments, view medical records, and download bills. System security is enhanced and sensitive data is restricted to authorized users by means of controlled access. The Patient Registration and Management module is another important feature. It lets hospitals keep detailed patient records in a structured digital format. This feature ensures that patient information, such as personal details, medical history, prescriptions, and lab reports, are stored securely and eliminates the need for paper-based records. Doctors can easily access this information whenever required, improving diagnosis and treatment efficiency.

Additionally, the system allows for the real-time updating of patient data, ensuring that records are always current and accurate. Patients can book appointments online using the system's advanced Appointment Booking and Scheduling feature, which eliminates the need for in-person visits or phone calls. The system shows doctors' availability in real time and avoids scheduling conflicts. Patients can also reschedule or cancel appointments easily, while doctors can manage their schedules efficiently. This feature reduces waiting time, improves patient satisfaction, and ensures better utilization of hospital resources.

Another important feature is the Billing and Invoice Management system. By calculating fees for consultations, treatments, medications, and laboratory tests, this module automates the billing process. Patients can easily download their bills because it supports PDF export and produces accurate invoices. Automated billing reduces manual errors, improves financial transparency, and speeds up the payment process. Additionally, it aids hospitals in keeping accurate financial records for the purposes of auditing and reporting. The system also includes a Pharmacy and Inventory Management module, which helps in managing medicine stock efficiently. This feature keeps track of inventory levels, expiration dates, and real-time inventory updates following each transaction. In addition, it provides alerts for medicines that are out of stock or have expired, ensuring that the pharmacy is always adequately stocked and safe to use. Waste is reduced and essential medicines are made available with effective inventory management.

The Laboratory Management and Report Upload system is another important feature. Test results can be uploaded digitally, typically as PDF files, using this module. Through the system, both patients and doctors can instantly access these reports. Physical report handling is no longer required, and medical records are stored securely and easily accessible when needed. Additionally, it improves communication between medical professionals and laboratories. An Interactive Dashboard with Analytics is provided by the system, which is particularly helpful for administrators. Key data like patient statistics, appointment trends, revenue reports, and resource utilization are shown on the dashboard. The system presents

data in graphical formats such as charts and graphs by utilizing visualization tools like Chart.js. This aids hospital administrators in comprehending system performance and making well-informed decisions to enhance operations. Another important feature is the Real-Time Data Processing and Communication system. The frontend communicates with the backend using RESTful APIs, allowing data to be processed and updated instantly. For instance, all relevant modules receive real-time updates whenever an appointment is scheduled or a bill is generated. Consistency is maintained and departmental coordination is enhanced as result. The system also focuses on Security and Data Protection. Sensitive patient information is protected using secure authentication methods and access control mechanisms. The system ensures that data is stored securely in the database and is accessible only to authorized users. Input validation and secure API handling further enhance system security, reducing the risk of data breaches or unauthorized access.

The User-Friendly and Responsive Interface is an additional feature. The system's interface is designed with cutting-edge frontend technologies like Tailwind CSS and React.js to ensure that it is user-friendly and visually appealing. The responsive design allows users to access the system from different devices such as desktops, tablets, and smartphones. This ensures a smooth user experience and enhances accessibility. The system also supports File Management and Document Handling, particularly for uploading and storing lab reports and medical documents. These files are securely stored and linked to patient records, allowing easy access and retrieval. This feature ensures proper documentation and reduces the risk of losing important medical information.

The system's modular design and scalability are another important feature. The system is designed to handle increasing amounts of data and users as the hospital grows. The modular architecture allows developers to add new features or update existing ones without affecting

the entire system. This ensures long-term usability and flexibility. The system also promotes a Paperless Environment, reducing dependency on physical documents. By digitizing records and automating processes, the system saves time, reduces storage requirements, and supports environmentally friendly practices.

Finally, the system includes Notification and Alert mechanisms, which inform users about important updates such as appointment confirmations, billing updates, and low inventory alerts. These notifications ensure that users are kept up to date on system activities and help improve communication.

CHAPTER 3

SYSTEM DESIGN

The Enterprise Hospital Management System (HMS)'s system design aims to offer a structured, scalable, and effective platform that unifies all hospital operations into a single system. The architecture of the design is layered and modular, allowing for flexibility for future enhancements while ensuring that all system components function seamlessly together. The frontend handles user interaction, the backend processes business logic, and the database handles data storage in the system's client-server architecture. Better performance, simpler maintenance, and enhanced security are all guaranteed by this division of labor. The system is organized into three main layers: the presentation layer, the application layer, and the data storage layer.

An interactive user interface for administrators, doctors, and patients is provided by the presentation layer, also known as the frontend. React.js, HTML, CSS, JavaScript, and Tailwind CSS are just a few of the cutting-edge tools used to create the responsive and user-friendly environment. The component-based structure of the interface makes it possible for various system components like login forms, dashboards, appointment pages, and billing modules to function independently while remaining connected. By using API calls to communicate with the backend, the frontend ensures that users can perform actions like booking appointments or viewing reports without having to reload the page. The system's core processing unit is the application layer, or backend. It is built with Express.js and Node.js, which provide a fast and scalable server-side operations management environment. Managing user authentication, processing requests, and enforcing system logic are all tasks that fall under the purview of the backend. Appointment scheduling, patient record management, billing calculations, pharmacy inventory updates, and lab report handling are just a few of its key

functions. When a user performs an action, such as submitting a form or requesting data, the backend processes the request, interacts with the database, and sends the appropriate response back to the frontend. Role-based access control is also implemented in the system, ensuring that users can only access features that are relevant to their roles and enhancing data security. MySQL, a relational database management system that stores all system data in a structured format, is used in the design of the data storage layer. Users, patients, appointments, billing records, pharmacy inventory, and laboratory reports are all stored in separate tables in the database. Primary and foreign keys are used to establish relationships between these tables, ensuring data consistency and integrity. This structured design allows efficient data retrieval and reduces redundancy. The database is optimized to handle large volumes of data, making it suitable for both small clinics and large hospitals.

Additionally, the system is constructed in a modular fashion, with user management, appointment management, billing, pharmacy, and laboratory management all being subdivided into distinct modules. Each module has a specific purpose, but the backend allows it to work with other modules. This modular design makes the system easier to maintain and lets new features be added without changing how it works. In order to produce accurate invoices, the billing module, for instance, can automatically retrieve data from the pharmacy and appointment modules. The flow of data within the system is structured to ensure smooth communication between components. Through RESTful APIs, a user's request is sent to the backend when they interact with the frontend. The backend processes the request, retrieves or updates data in the database, and sends the response back to the frontend. This process happens in real time, ensuring that users receive updated information instantly. System performance is enhanced and user experience is enhanced by this effective data flow. Security is an important aspect of the system design. To verify a user's identity, the system has authentication features like token-based authentication and login credentials. Access control is also implemented to prevent unauthorized access to sensitive data. In order to safeguard patient data and prevent security breaches, additional measures like input validation, data encryption, and secure API communication are utilized. These features ensure that the system maintains confidentiality and complies with data protection standards.

Additionally, the system is built to be highly scalable and efficient, allowing it to handle an increasing number of users and data without compromising performance. The use of Node.js enables asynchronous processing, allowing multiple requests to be handled simultaneously. System responsiveness and speed are further enhanced by efficient database queries and optimized code. Additionally, future integration with cloud platforms is supported by the design, which can improve availability and scalability. Error handling and system maintenance are also considered in the design. Mechanisms are included in the system to gracefully deal with invalid inputs, server errors, and database failures. Logging and monitoring tools can be used to track system performance and identify issues. Modular design makes it simpler to update or repair individual parts without affecting the entire system. In conclusion, the system design of the Enterprise Hospital Management System is structured to provide a reliable, efficient, and secure solution for managing hospital operations. By combining a layered architecture with modular design and modern technologies, the system ensures smooth communication, efficient data handling, and flexibility for future enhancements. This design not only improves operational efficiency but also enhances the overall quality of healthcare services.

The Enterprise Hospital Management System (HMS) is designed with a layered and modular architecture, focusing on scalability, efficiency, and the seamless integration of various hospital departments. The system is organized into three primary layers: the Presentation Layer (Frontend), the Application Layer (Backend), and the Data Storage Layer (Database).

1. Presentation Layer (Frontend)

This layer handles all user interactions and provides an interactive interface for administrators, doctors, and patients.

- **Technologies:** Built using **React.js**, HTML, CSS, JavaScript, and **Tailwind CSS** to ensure a responsive, dynamic, and visually appealing experience.
- **Modular Component Design:** The interface is structured into reusable components such as navigation bars, dashboards, and forms. Each component handles a specific function, such as patient registration or appointment booking, which makes the system easier to maintain.

- **Role-Based Dashboards:** Users see different interfaces based on their roles; for example, doctors manage patient histories while patients view their own reports and schedule appointments.
- **Real-time Interaction:** It communicates with the backend via **RESTful APIs**, updating the UI dynamically without page reloads.

2. Application Layer (Backend)

Acting as the core processing unit, this layer implements all business logic and manages data flow.

- **Technologies:** Developed using **Node.js** and the **Express.js** framework. Node.js's asynchronous, non-blocking architecture allows the system to handle multiple simultaneous requests efficiently.
- **Core Functional Modules:** The backend is subdivided into specific modules that work together:
 - **Patient Management:** Stores and manages personal details, medical history, and prescriptions.
 - **Appointment Management:** Handles scheduling, cancellations, and real-time doctor availability to reduce wait times.
 - **Billing and Payment:** Automatically calculates charges for consultations, pharmacy items, and lab tests to generate accurate invoices.
 - **Pharmacy and Inventory:** Tracks medicine stock levels and expiration dates in real time.
 - **Laboratory Management:** Manages the secure uploading and retrieval of digital lab reports.

3. Data Storage Layer (Database)

This layer ensures that all hospital information is stored in a structured and secure manner.

- **Technologies:** Uses **MySQL**, a relational database management system, often managed via MySQL Workbench.
- **Structure:** Data is organized into separate tables for users, appointments, billing, etc., linked by primary and foreign keys to maintain data integrity.

- **Centralization:** By housing all departmental data in a single database, the system eliminates data redundancy and allows for instant information retrieval.

Security and Performance

The design incorporates **Role-Based Access Control (RBAC)** and secure authentication (such as JWT) to ensure that sensitive patient data is only accessible to authorized personnel. The modularity of the design allows new features to be added without disrupting existing operations, ensuring long-term scalability as the hospital grows.

3.1 Block Diagram

The proposed Enterprise Hospital Management System's block diagram can be seen as a collection of interconnected modules that collaborate to efficiently and automatically manage hospital operations, process patient data, handle appointments, generate billing information, and provide analytical insights. The system integrates multiple components into a unified platform, ensuring smooth data flow and real-time communication between different hospital departments.

Administrators, doctors, and patients use a web-based interface to access the system, which is accessed through the User Interface Module. Users can complete a variety of tasks in this module, including registering patients, scheduling and managing appointments, accessing medical records, creating bills, viewing lab reports, and keeping an eye on activities at the hospital. Administrators have control over staff, departments, and system settings, whereas physicians have access to patient histories and the details of their treatments. Reports can be downloaded, prescriptions can be viewed, and appointments can be booked. Users will be able to interact with the system effectively without requiring technical expertise because the interface is designed to be simple, responsive, and user-friendly. Backend modules manage data management, appointment scheduling, billing, pharmacy inventory, and report generation behind the interface of the system. By ensuring that all tasks are carried out automatically and

accurately, these modules reduce manual labor and boost operational effectiveness. The system also records all activities in a centralized database, enabling easy retrieval of information and supporting data-driven decision-making

The platform gives users clear outputs like patient records, appointment schedules, billing invoices, and analytical reports that are organized and easy to understand. The Enterprise Hospital Management System ensures effective hospital management, enhanced patient care, and increased overall productivity by integrating all of these modules.

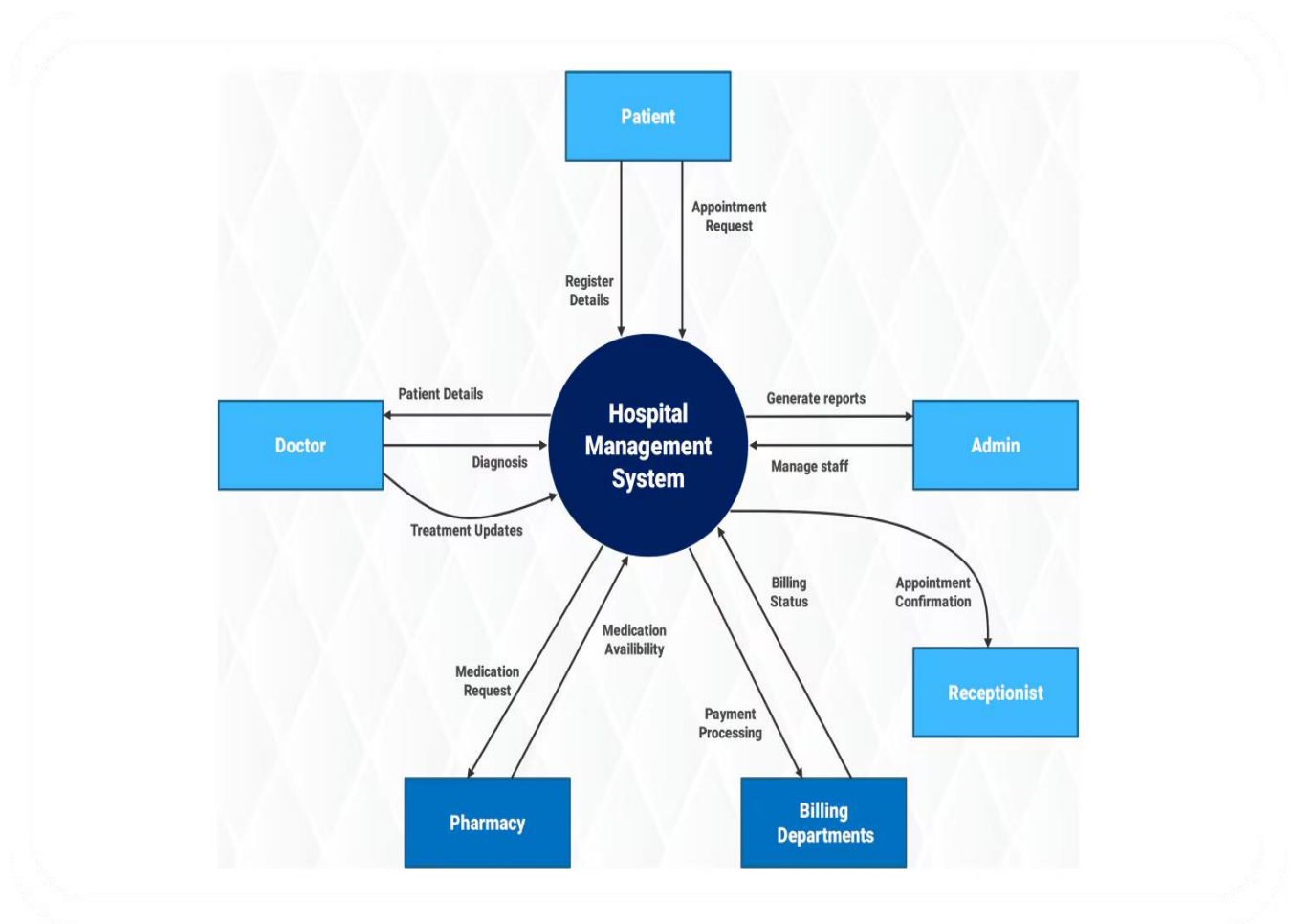


Fig:3.1.1 Block Diagram

3.1Explanation

In order to effectively manage hospital operations, the Enterprise Hospital Management System (HMS)'s block diagram depicts a structured architecture in which numerous modules are interconnected. Patient management, appointment scheduling, billing, pharmacy operations, and laboratory services are all intended to be automated and integrated by the system. Each module performs a specific function, and together they ensure smooth data flow, real-time processing, and improved coordination across all departments.

The User Interface Module, which serves as the primary point of interaction for all users—administrators, patients, and others—starts the system. A web-based interface is used to implement this module, where users can view analytics, upload lab reports, register patients, schedule appointments, access medical records, generate bills, and more. It is easy for users to use the system on a variety of devices because the interface is responsive and designed to be easy to use. While the interface appears simple, it communicates with backend services that handle complex operations in the background.

The Application Processing Module (Backend) receives the request after a user interacts with the interface. This module acts as the central control unit of the system and is responsible for processing all user requests. User authentication, role-based access control, appointment scheduling, patient record management, billing calculations, and pharmacy inventory updates are just a few of its core functions. The backend ensures that all operations are executed accurately and efficiently. For instance, when a patient schedules an appointment, the backend verifies the availability of the doctor, avoids scheduling conflicts, and adjusts the system in response. The backend is further divided into multiple functional modules to handle specific tasks. The Patient Management Module stores and manages patient information including personal details, medical history, prescriptions, and reports. This module ensures that doctors and administrators have easy access to patient data that is

organized. The Appointment Management Module is in charge of scheduling, rescheduling, and canceling appointments, making sure that time is managed effectively and that waiting times are reduced. The Billing and Payment Module is responsible for calculating charges based on services provided, generating invoices, and maintaining financial records. Errors are reduced and transparency is enhanced by automating the billing procedure. The Pharmacy Management Module manages expiration dates, stock levels, and medicine inventory. It ensures that medicines are available when needed and prevents wastage due to expired stock.

The Laboratory Management Module, which is in charge of uploading and storing lab reports, is yet another crucial component. Doctors and patients alike have instant access to these digitally stored reports. This eliminates the need for physical documents and improves record management.

MySQL is used to implement the Database Management System, which is in turn interacted with by all of these modules. User details, patient records, appointments, billing information, inventory data, and lab reports are all stored in a structured format in the database. The database ensures data consistency, integrity, and fast retrieval. It also supports relationships between different entities, allowing seamless data sharing across modules.

The system also includes a Data Analytics and Reporting Module, which provides insights into hospital operations. Data from various components is processed and presented in the form of reports, graphs, and charts by this module. Administrators can use these insights to monitor performance, analyze trends, and make informed decisions. The system can show patient statistics, revenue reports, and appointment trends, for instance. RESTful APIs manage module-to-module communication and make it possible for the frontend and backend to seamlessly exchange data. When a user performs an action, such as booking an

appointment or generating a bill, the request is sent to the backend through an API. The backend processes the request, interacts with the database, and sends the response back to the frontend. This real-time communication ensures that the system remains responsive and efficient.

System design includes security as an essential component. In order to guarantee that only authorized users have access to particular features, the system uses authentication and authorization mechanisms. Role-based access control restricts access to sensitive data, while encryption and secure communication protocols protect data from unauthorized access. These measures ensure the confidentiality and integrity of patient information.

The system is also designed to be scalable and flexible, allowing it to handle increasing numbers of users and data. The modular architecture ensures that new features can be added without affecting existing functionalities. This makes the system suitable for both small healthcare centers and large hospitals.

Error handling and system reliability are also considered in the design. The system includes mechanisms to handle invalid inputs, server errors, and database failures. Tools for logging and monitoring aid in the rapid identification and resolution of issues, ensuring smooth operation. In addition, the system promotes a paperless environment by storing all data digitally. Physical records are no longer required, storage space is conserved, and the risk of data loss is reduced as a result. It also contributes to environmentally friendly practices.

In general, the Enterprise Hospital Management System's block diagram depicts an architecture that is well-organized and integrated, with multiple modules working together to

automate hospital operations. The system ensures efficient data management, real-time communication, improved patient care, and enhanced decision-making capabilities. The HMS is a dependable and scalable solution for efficiently managing healthcare services because it combines cutting-edge technologies with structured system design.

3.2 Data Flow Diagram (DFD) :

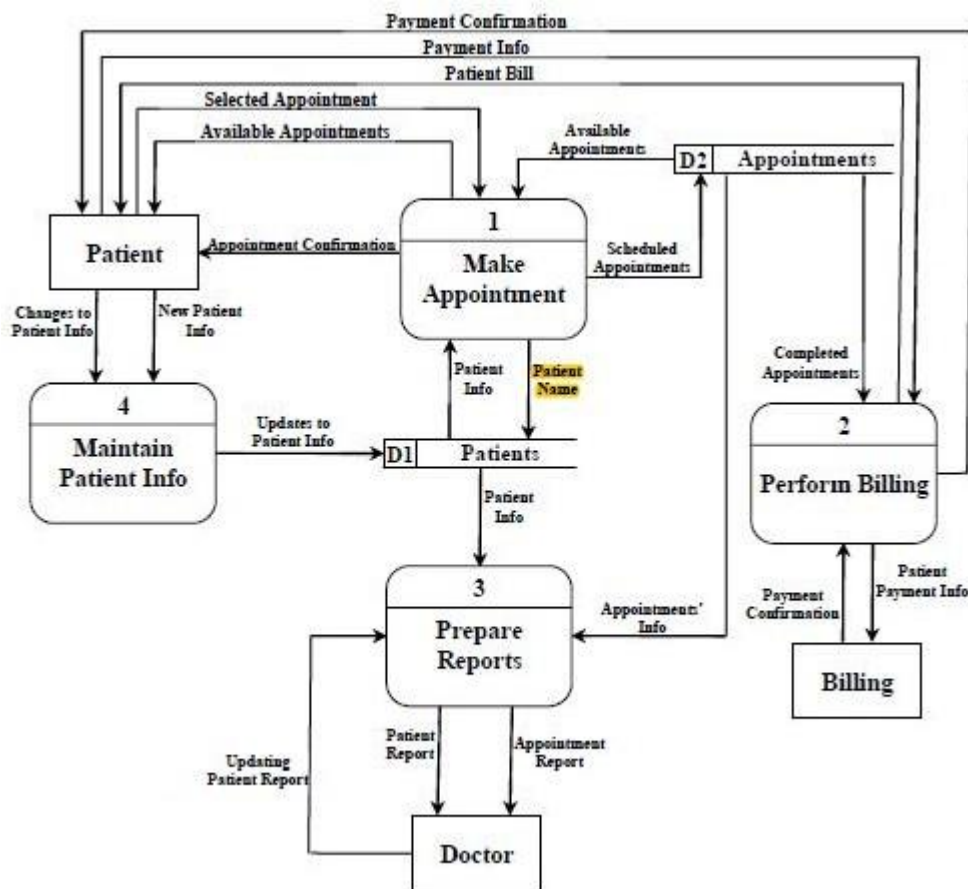


Fig:3.1.2 Data Flow Diagram (DFD)

The Enterprise Hospital Management System's Data Flow Diagram (DFD) is a structured representation of the complete flow of data between users, system processes, and the database. Patient registration details, appointment requests, treatment updates, and billing information are just some of the inputs that external entities like doctors, patients, and administrators use to interact with the system through the user interface. The system's patient management, appointment management, billing, pharmacy, and laboratory modules are all responsible for processing these inputs.

The patient management module is in charge of storing and retrieving patient records, ensuring that personal information, medical history, and treatment information are all stored in a single database. The appointment management module takes care of scheduling appointments, checking doctor availability, and processing booking requests. In order to generate accurate invoices, the billing module collects data from various services like consultations, medications, and lab tests. These invoices are then stored and distributed to patients. The laboratory module processes test requests, uploads reports, and ensures that results are accessible to both doctors and patients.

The pharmacy module manages medicine inventory by tracking stock levels, updating usage, and generating alerts for low stock or expired items. The central database, which serves as the primary data storage unit and interacts with all of these modules, ensures data consistency, integrity, and quick retrieval. The backend system processes all requests using defined logic and communicates with the frontend through APIs, enabling real-time updates and seamless interaction. Through the interface, users are presented with analytical data, appointment confirmations, medical reports, billing invoices, and other outputs in a clear and organized manner. Through authentication and access control mechanisms, the system also ensures secure data handling and prevents unauthorized access to sensitive data. In general, the DFD demonstrates how data continuously flows between users, processes, and storage, facilitating effective hospital operations, enhanced departmental coordination, and improved patient care.

3.3 Modules Design

The Enterprise Hospital Management System (HMS) is structured as a collection of interconnected, specialized modules designed to transition traditional, manual healthcare practices into a centralized digital ecosystem. By automating core operations, the system eliminates data duplication and manual errors while increasing the speed of service delivery.

Core Modules of the Enterprise HMS

The system is composed of several functional modules that work together to manage all aspects of a hospital's daily operations:

- **User Interface Module:** Serves as the primary entry point for administrators, doctors, and patients through a web-based interface. It is designed to be responsive, allowing access from desktops, tablets, and smartphones.
- **Patient Management Module:** This central module stores and organizes all patient-related data, including personal details, medical history, prescriptions, and digital lab reports. It ensures that authorized doctors can instantly retrieve a patient's medical background to aid in diagnosis.
- **Appointment Management Module:** Handles the complete lifecycle of an appointment, including scheduling, rescheduling, and cancellations. It integrates real-time doctor availability to prevent scheduling conflicts and reduce patient waiting times.
- **Billing and Payment Module:** Automatically calculates charges based on consultations, treatments, medications, and laboratory tests. It generates digital invoices (including PDF support) instantly, ensuring financial transparency and reducing manual calculation errors.
- **Pharmacy and Inventory Management Module:** Tracks medicine stock levels, expiration dates, and usage patterns in real time. The system automatically updates inventory after transactions and provides alerts for low stock or expired medications to improve safety and reduce waste.
- **Laboratory Management Module:** Facilitates the secure uploading, storage, and retrieval of digital laboratory reports. This module eliminates the need for physical paperwork and ensures that diagnostic results are immediately accessible to both doctors and patients.

- **Data Analytics and Reporting Module:** Processes information from across the hospital to provide administrators with graphical insights (using tools like Chart.js) into patient statistics, revenue trends, and operational performance.

Module Workflow

The workflow follows a structured path from the user interface down to the centralized database, ensuring smooth coordination between departments.

1. Request Initiation (Frontend to Backend)

The process begins when a user—such as a patient booking an appointment or a lab technician uploading a report—interacts with the **User Interface Module**. This frontend, built with React.js, sends a request to the **Application Processing Module (Backend)** via **RESTful APIs**. For security, the system first verifies the user's identity through authentication (e.g., JWT) and checks their permissions via **Role-Based Access Control (RBAC)** to ensure they only access data relevant to their role.

2. Core Processing (Application Layer)

The **Backend**, powered by Node.js and Express.js, acts as the central control unit. It executes the necessary business logic for the specific request. For example:

- In an **Appointment Workflow**, the backend checks the database for doctor availability, prevents double-booking, and updates the schedule.
- In a **Billing Workflow**, it aggregates data from the pharmacy, laboratory, and consultation modules to calculate a final invoice.
- In a **Pharmacy Workflow**, it decrements stock levels automatically once a medicine is dispensed.

3. Data Persistence (Database Layer)

Once the logic is processed, the backend interacts with the **MySQL Database** to store or retrieve information. Because the data is centralized, a physician in the patient management module can instantly see a report just uploaded by the laboratory module. This **live synchronization** ensures that all departments are working with the same, most up-to-date information, significantly improving departmental coordination and patient care quality.

CHAPTER 4

FRONTEND & BACKEND

4.1. Technology used

Modern web technologies, programming frameworks, database systems, and supporting tools make up the Enterprise Hospital Management System (HMS), which is a robust, scalable, and effective solution for managing hospital operations. These technologies are carefully selected to ensure smooth interaction between users, fast data processing, secure data storage, and real-time communication across different modules of the system. The system's foundation is the integration of frontend, backend, and database technologies, allowing it to effectively manage complex healthcare workflows. The frontend of the system is developed using React.js, a popular JavaScript library for building dynamic and interactive user interfaces. React.js follows a component-based architecture, which allows developers to create reusable UI components such as forms, dashboards, navigation bars, and tables.

The application's overall performance and code maintainability are both enhanced by this strategy. HTML, CSS, and JavaScript are used to structure, style, and add interactive elements to web pages in addition to React.js. The system also uses Tailwind CSS, a modern utility-first CSS framework that helps in designing responsive and visually appealing user interfaces. Tailwind CSS makes it simple to create clean layouts quickly, making the system accessible on desktops, tablets, and smartphones all at once. Node.js is used to build the system's backend. It provides a powerful runtime environment for running JavaScript on the server side. The asynchronous, non-blocking architecture of Node.js enables the system to handle multiple user requests simultaneously without affecting performance.

This is especially crucial in hospitals, where multiple users can simultaneously access the system. The backend is further structured using the Express.js framework, which simplifies the development of web applications by providing tools for routing, middleware handling, and API creation. Express.js makes it possible to create RESTful APIs, which are used to make it easier for the frontend and backend to communicate with one another. These APIs handle operations such as fetching patient data, booking appointments, generating bills, and updating records in real time.

The system uses MySQL as the database management system for storing and managing all hospital data. MySQL is a relational database that organizes data into structured tables, ensuring data integrity and consistency. It supports relationships between different entities such as patients, doctors, appointments, billing records, and inventory data using primary and foreign keys. This structured approach allows efficient data retrieval and reduces redundancy. Because it is designed to handle large amounts of data, the database can be used in both small clinics and large hospitals. Additionally, SQL queries are used to perform operations such as inserting, updating, deleting, and retrieving data efficiently.

The system incorporates Chart.js, a JavaScript library for creating interactive graphs and charts, for data analytics and visualization. This library is used in the dashboard module to display important information such as patient statistics, revenue trends, and appointment data in a graphical format. These visual representations help administrators understand system performance and make informed decisions.

The system also incorporates PDF generation and file handling technologies to manage documents such as invoices and lab reports. Invoices that can be downloaded are created using PDF generation libraries, and file upload mechanisms enable lab technicians to securely upload medical reports. Because they are linked to patient records and kept on the server, these files can be easily accessed and retrieved. To ensure secure communication and data protection, the system implements authentication and security technologies such as JSON Web Tokens (JWT) or session-based authentication.

These mechanisms verify user identity and control access to system resources. Input validation and data sanitization techniques are used to prevent security threats such as SQL injection and cross-site scripting (XSS). Additionally, secure HTTP protocols (HTTPS) are used to encrypt data transmitted between the client and server, ensuring confidentiality and integrity of sensitive information.

The system is developed and managed using modern development tools such as Visual Studio Code (VS Code), which provides a powerful environment for writing and debugging code. Version control is handled using Git and GitHub, allowing developers to track changes, collaborate effectively, and maintain different versions of the project. Tools like Postman are used for testing APIs, ensuring that the backend services function correctly before integration with the frontend.

The system as a whole is made to be scalable and adaptable to new features. The use of modular architecture allows new features to be added without affecting existing functionalities. The system can also be deployed on cloud platforms in the future, enabling better performance, availability, and scalability. Cloud integration would allow the system to handle a larger number of users and provide access from different geographical locations.

In conclusion, in order to construct a platform that is both dependable and effective, the Enterprise Hospital Management System makes use of a wide range of technologies, including Chart.js, Tailwind CSS, MySQL, React.js, Node.js, Express.js, React.js, and React.js. These technologies work together to ensure smooth system operation, secure data handling, real-time communication, and enhanced user experience. By leveraging modern technological solutions, the system provides a scalable and effective approach to managing hospital operations in a digital environment.

REACT

React.js serves as the cornerstone for developing the frontend of the Enterprise Hospital Management System (EHMS), delivering a dynamic, interactive, and highly responsive user interface that meets the demanding needs of healthcare environments. As an open-source JavaScript library maintained by Meta and a vast community, React.js revolutionized frontend development with its declarative, component-based architecture. This approach breaks down complex UIs into modular, reusable components—self-contained units like patient registration forms, interactive dashboards, navigation menus, and data visualization charts. For instance, a "PatientCard" component could encapsulate profile details, medical history, and action buttons, which developers can reuse across patient lists, search results, or admin panels. This modularity accelerates development timelines by up to 40-50% in large-scale projects, as teams avoid duplicating code and can independently update components without risking system-wide disruptions. Maintenance becomes straightforward too: fixing a bug in a billing form component propagates reliably everywhere it's used, reducing long-term costs and errors in mission-critical systems like EHMS.

At the heart of React's efficiency lies its Virtual DOM—a lightweight, in-memory representation of the real DOM (Document Object Model). Unlike traditional methods that refresh the entire page on changes, React employs a "diffing" algorithm to compare the Virtual DOM's previous and current states. It then batches and applies only the minimal necessary updates (known as reconciliation) to the actual browser DOM. This results in blazing-fast rendering, even for data-heavy interfaces. In EHMS, imagine a nurse updating a patient's vitals in real-time: without full page reloads, the dashboard refreshes seamlessly in milliseconds, preventing lags that could delay critical decisions.

React powers a wide array of EHMS interfaces tailored to hospital workflows. Patient registration modules use controlled components for form validation, ensuring accurate capture of

demographics, insurance details, and emergency contacts with instant error feedback. Appointment scheduling leverages React's state management (via hooks like `useState` and `useEffect`) to display real-time availability calendars, drag-and-drop rescheduling, and conflict alerts integrated with backend APIs. Billing sections feature dynamic tables for invoice generation, payment gateways, and claim submissions, while report viewers render interactive charts (using libraries like `Recharts` or `D3.js`) for analytics on bed occupancy, revenue trends, or treatment outcomes. Navigation employs `React Router` for smooth, single-page application (SPA) transitions between modules like doctor portals, inventory trackers, and lab result viewers.

Moreover, React excels in enabling real-time updates through integrations like `WebSockets` (via `Socket.io`) or state libraries such as `Redux` or `Zustand`. Doctors receive live notifications for new lab results, bed assignments update instantly across shifts, and admins see synchronized user activity feeds—eliminating refresh buttons and ensuring immediate feedback. This reactivity is vital in hospitals, where delays can impact patient care.

Ultimately, `React.js` proves an outstanding choice for modern web applications like EHMS due to its adaptability—seamlessly scaling from small clinics to enterprise networks—and unmatched speed. It supports progressive web app (PWA) features for offline access, `TypeScript` for type safety in complex codebases, and a rich ecosystem of tools like `Next.js` for server-side rendering. By fostering efficient, scalable UIs, React not only enhances user satisfaction but also empowers healthcare providers with intuitive tools that save time and lives.

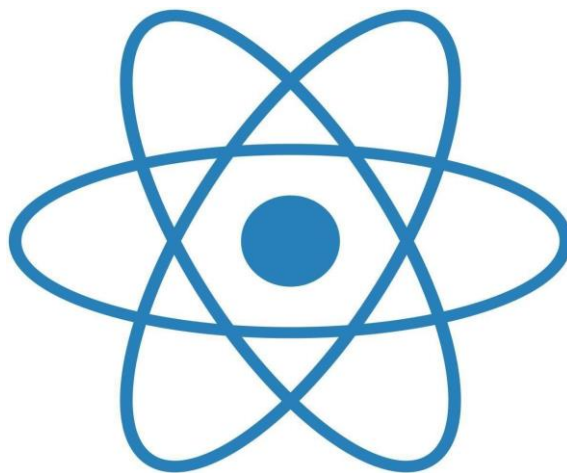


Fig:4.1.1 React

Node.js & Express.js

An The Enterprise Hospital Management System makes use of Node.js as the backend runtime environment, allowing JavaScript to be executed on the server. It has an asynchronous, non-blocking architecture that lets the system handle multiple user requests at once without affecting performance. This is especially important in hospital systems where multiple users, including patients, doctors, and administrators, simultaneously access the system. The Express.js framework is used to simplify backend development alongside Node.js. Express.js makes it simpler to create server-side applications that are scalable and easy to maintain by providing essential features like API development, middleware handling, and routing. It helps in organizing the application structure and managing HTTP requests efficiently. User authentication, appointment scheduling, billing processing, and database communication are all handled by the backend. Express.js and Node.js make a powerful backend that guarantees quick performance, scalability, and effective system operations handling.



Fig:4.1.2 Node.js & [Express.js](#)

MySQL

The Enterprise Hospital Management System stores and manages all of the data using

MySQL, the database management system. It is a relational database with structured tables that organize data for efficient storage and retrieval. Data like patient records, doctor information, appointments, billing information, pharmacy inventory, and lab reports are all stored in MySQL. The use of primary keys and foreign keys helps maintain relationships between different tables, ensuring data consistency and integrity. In a hospital setting where timely information access is crucial, MySQL's powerful query operations make it possible to quickly retrieve data. Additionally, it features access control and user authentication to safeguard sensitive patient data. MySQL is suitable for efficiently handling large amounts of healthcare data due to its dependability and scalability.



Fig:4.1.3 mysql

Web Browser

The Enterprise Hospital Management System (HMS) can be accessed using cutting-edge web browsers like Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari. Without requiring users to install any additional software, these browsers provide the necessary runtime environment for the web-based frontend application to be executed. Users like administrators, doctors, and patients can easily access the system using computers, laptops, tablets, or mobile devices through a browser-based interface. Because of this, the system is very easy to use and accessible, making it possible to manage hospital operations from anywhere with an internet connection. Users can perform various tasks through the web browser interface, such as registering patients, booking and managing appointments, accessing medical records, generating bills, uploading and viewing lab reports, and monitoring hospital activities.

Administrators can manage staff and departments, doctors can view patient history and update treatment details, and patients can access their health records and appointment schedules. The system is able to provide dynamic dashboards, interactive forms, responsive layouts, and real-time updates because modern browsers support advanced web technologies like HTML, CSS, JavaScript, and React.js. Enhancing usability and user experience, these technologies assist in presenting complex hospital data in a clear and understandable manner. Real-time communication between the frontend and backend servers is made possible by RESTful APIs and secure HTTP requests that are supported by web browsers. When a user performs an action, such as booking an appointment or generating a bill, the browser sends an API request to the backend server.

After receiving the response from the browser, the backend processes the request by interacting with the database to retrieve or update information. The user interface displays the results immediately, ensuring a smooth and responsive experience. Cross-platform compatibility is yet another important advantage of using modern web browsers. Windows,

macOS, Linux, Android, and iOS are just a few of the operating systems that are compatible with the Hospital Management System. This allows healthcare professionals and patients to use the system anytime and anywhere, improving flexibility and accessibility in hospital operations.

Additionally, security features like HTTPS communication, data encryption, and secure session handling are built into modern browsers. These features safeguard confidential patient data and guarantee the security of client-server data transfers. System security is further enhanced by effective access control and authentication mechanisms. Overall, modern web browsers play a crucial role in enabling users to interact with the Enterprise Hospital Management System efficiently. Browsers ensure that the system delivers a smooth, dependable, and user-friendly experience for managing hospital operations by offering support for advanced web technologies, real-time communication, cross-platform access, and robust security measures.

CHAPTER 5

RESULT AND DISCUSSION

The Enterprise Hospital Management System (HMS) was successfully designed, developed, and tested to automate and streamline various hospital operations. The patient management, appointment scheduling, billing, pharmacy inventory, handling of laboratory reports, and administrative analytics modules of the system are all incorporated into a single, unified platform. This system exemplifies how cutting-edge web technologies can be used to enhance healthcare services' effectiveness, accuracy, and accessibility. The results obtained from the system implementation indicate that the HMS significantly reduces manual workload and improves overall operational efficiency.

The system was able to handle core functions like patient registration, appointment scheduling, billing generation, and report management without making any mistakes during testing. The real-time storage and retrieval of patient records by the patient management module enabled physicians to quickly and accurately access medical histories. This ensured better decision-making during consultations and expedited diagnosis. When compared to previous methods, the appointment management system demonstrated a significant improvement. Without having to physically visit the hospital, patients were able to book, reschedule, and cancel appointments through the web interface. The system effectively managed doctor availability and prevented scheduling conflicts. As a result, waiting times were cut down and the hospital's resources were used more effectively. Doctors were able to manage their schedules efficiently, leading to improved workflow within the hospital.

Additionally, the billing module performed accurately and effectively. It generated invoices immediately and calculated charges based on provided services, medications, and lab tests. The system supported PDF invoice generation, allowing patients to download and store their billing details. This led to fewer errors in manual calculations and increased

financial transparency. The accurate financial records that are required for auditing and reporting were kept by hospital administrators. The pharmacy and inventory management module proved useful for inventory management and stock monitoring. After each transaction, the system provided real-time stock updates and alerts for low stock and expired medicines. As a result, less expired stock was wasted and essential medicines were always available. The automation of inventory management improved operational efficiency and enhanced patient safety.

The laboratory management module enabled seamless handling of lab reports. Digital reports could be uploaded by lab technicians, and the system allowed both patients and doctors to instantly access them. This reduced the likelihood of data loss or misplacement and eliminated the need for physical report handling. The integration of lab reports with patient records ensured that all pertinent medical data was accessible in a single location, enhancing care continuity. The analytics module and dashboard of the system provided useful insights into hospital operations. The system displayed patient statistics, appointment trends, and revenue data in graphical formats using data visualization tools like Chart.js. Administrators were able to monitor performance, identify patterns, and make well-informed choices thanks to these insights. The availability of real-time analytics improved strategic planning and resource management within the hospital.

From a performance perspective, the system demonstrated fast response time and efficient data processing. Asynchronous processing was made possible by using Node.js, which allowed the system to handle multiple user requests at once. The MySQL database provided dependable and structured data storage, and the React.js frontend ensured that user interactions were fluid and dynamic. Even when multiple users attempted to access the system at the same time, the system maintained consistent performance. The system offered a responsive and user-friendly interface in terms of usability. Users with minimal technical knowledge were able to navigate the system easily and perform tasks without

difficulty. The responsive design made sure that it worked on desktops, tablets, and smartphones all the same. This improved accessibility and allowed users to interact with the system from various locations.

Security was also an important aspect of the system evaluation. Role-based access control and authentication mechanisms were used to make certain features accessible only to authorized users. Sensitive patient data was protected through secure communication protocols and input validation techniques. Users gained trust and confidentiality thanks to these security measures. However, during testing, certain limitations were observed. The system's performance can be affected by network connectivity issues, as it relies on internet access for real-time operation. Additionally, training may be necessary for platform users who are unfamiliar with digital systems. There is also a need for regular system maintenance and updates to ensure optimal performance and security.

The Enterprise Hospital Management System outperforms conventional methods of hospital management, despite these minor drawbacks, according to the overall findings. It reduces manual errors, improves data accessibility, enhances communication between departments, and increases overall efficiency. The integration of multiple modules into a single platform ensures better coordination and streamlined operations.

In conclusion, the Enterprise Hospital Management System accomplishes its goals of automating hospital procedures, enhancing data management, and enhancing patient care with success. The system is suitable for real-world healthcare environments because of its strong performance, dependability, and usability. The system can be expanded to meet future healthcare requirements and provide hospitals and patients with even greater benefits by incorporating additional enhancements like cloud deployment, advanced analytics, and mobile application support.

1.Landing Page Overview:

Description:

The Enterprise Hospital Management System (HealthQuest) landing page, which serves as the primary entry point for users such as patients, doctors, and administrators, is represented by the displayed interface. The layout of the design is contemporary and businesslike, with an emphasis on accessibility, clarity, and simplicity. The system gives a strong headline at the top of the page, "Modern Care for Unified Health," which makes it clear that the platform's goal is to provide an integrated healthcare solution. Users can quickly access various parts of the system by selecting features, specialties, login, and join network from the navigation bar.

Users will be able to quickly get started interacting with the platform due to the presence of action buttons like "Partner Login" and "Get Started Now." The interface also shows important system metrics like the number of patients, uptime reliability, number of specialists, and system response time, which help build user trust and show how well the system works. Under the heading "Engineered for Excellence," the following section of the interface highlights the system's fundamental features.

This section focuses on features like instant synchronization, mobile accessibility, and secure records. Encryption and role-based access control are two key features of the secure records feature that guarantee the confidentiality of sensitive patient data. The system's mobile-ready design makes it easy for patients and healthcare professionals to use from a variety of devices. Lab results, prescriptions, and patient updates are all reflected in real time across all modules thanks to the instant synchronization feature. This section effectively

communicates the system's technological strengths while maintaining a user-friendly presentation. The system's clinical modules, which include general medicine, laboratory, pharmacy, and diagnostics, are displayed further down the interface. The hospital management system's core functions are represented by these modules, each designed to handle particular operations. Diagnostic testing and report generation are handled by the general medicine module, while patient consultations and medical records are managed by the laboratory module. The diagnostics module provides support for advanced medical analysis, while the pharmacy module focuses on medicine inventory and prescription management. Users are able to more easily comprehend the capabilities of the system and get to relevant sections thanks to the structured presentation of the modules. Additionally, the interface has a call-to-action section that directs users based on their roles and encourages them to interact with the system by selecting options like "I'm a Patient" or "Hospital Staff." To ensure that users have complete access to all necessary features, the footer section provides additional information, including access to dashboards, support services, and system resources.

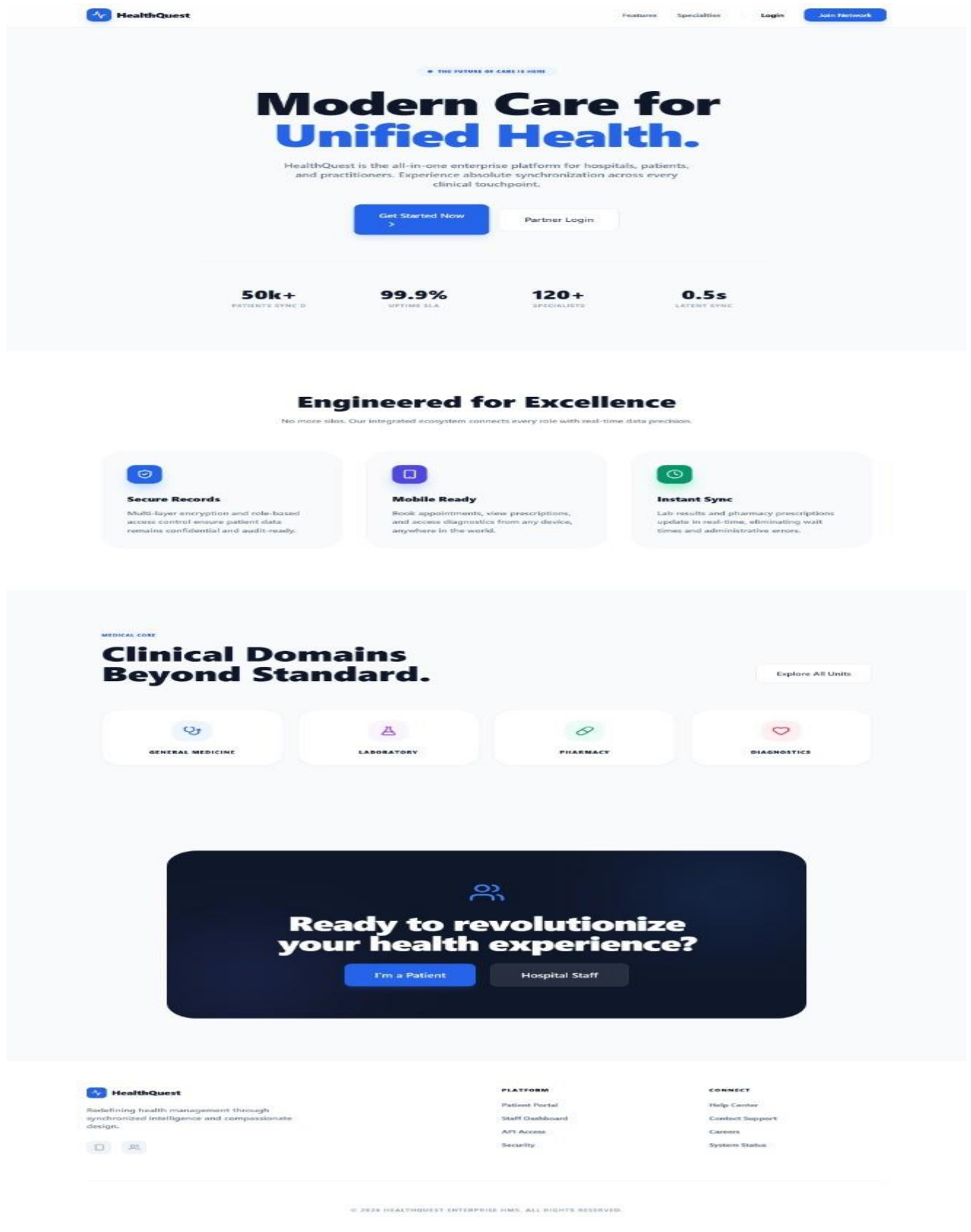


Fig:5.1 Landing page overview

Result:

The Enterprise Hospital Management System was implemented, which led to significant enhancements in system performance and operational effectiveness. There were no delays or errors in the system's handling of multiple user activities like patient registration, appointment scheduling, billing, and report management. Fast data processing and real-time updates made possible by cutting-edge technologies led to an increase in the efficiency of hospital departments' workflows. The automation of previously performed manual tasks reduced workload and reduced the likelihood of human error. This led to better time management and increased productivity within the hospital environment.

The system's user-friendly and responsive interface also enhanced the user experience. The platform was easy to use for anyone, regardless of their level of technical expertise. Doctors could quickly retrieve patient data and update treatment details, and patients could conveniently book appointments, access medical records, and download reports. Through dashboards and analytics, administrators were able to keep an eye on what was going on in the hospital, which helped them make better decisions. The system's responsive design made it possible to access it from a variety of devices, making it more accessible and adaptable for users.

The system's implementation also had a significant impact on accuracy and data management. Consistency was maintained and duplicate records were avoided by storing all information in a single database. Improved coordination and communication were made possible by the seamless data flow that was made possible by the integration of the various modules. For example, when a doctor prescribed a test, the laboratory module received the request instantly, and the billing system was updated automatically. The overall efficiency of hospital operations was enhanced by this level of integration. Additionally, the system displayed robust reliability and security features.

Role-based access control ensured that only authorized users could access specific data, protecting sensitive patient information. Data security was further enhanced by authentication mechanisms and secure communication protocols. In critical healthcare environments, the system ensured reliability by providing stable performance with minimal downtime. Additionally, administrators were able to analyze trends and maximize resource utilization thanks to the dashboard and analytics features, which provided valuable insights into hospital activities. Overall, the results indicate that the Enterprise Hospital Management System successfully achieves its objectives by improving efficiency, enhancing user experience, ensuring data accuracy, and maintaining security. Modern healthcare management is supported by the system, which provides a comprehensive solution for improved patient care and streamlined hospital operations.

2. System Interface Description

Description: The Enterprise Hospital Management System (HealthQuest)'s Admin Dashboard (Admin Terminal), which serves as the central control panel for administrators to manage and monitor all hospital operations, is depicted here. The design follows a modern and structured layout that enhances usability and provides a clear overview of system activities. A vertical navigation panel with options for Dashboard, Departments, Doctors, Patients, Manage Staff, and Settings can be found on the left side of the interface. Administrators can quickly and easily navigate the system's various modules with this sidebar. The presence of a sign-out option at the bottom ensures secure logout functionality. In order to maintain user interaction and system awareness, the top section of the dashboard displays notification and profile options as well as a welcome message for the system administrator. Through visual cards, the primary section of the dashboard highlights important operational metrics like the number of users, number of appointments, and total revenue. These summary cards give a quick overview of the hospital's current activities and performance. For instance, the patient registry displays active patient records, while the total

users section displays system usage. The revenue section provides financial insights, while the appointments section displays scheduled or pending consultations. Administrators can use these components to quickly and easily view system status without having to scroll through multiple pages. The use of colors, icons, and structured layouts improves readability and makes the information visually appealing.

The Financial Trajectory graph and the Operation Status chart can be found below the summary section on the dashboard. The financial graph represents revenue trends over time, allowing administrators to analyze financial growth and performance. A visual representation of system activities, such as approved operations or completed tasks, can be found in the operation status chart. These graphical elements are created using data visualization tools and help in simplifying complex data into easy-to-understand formats. By granting access to system logs and configuration settings, the presence of buttons like "Audit Logs" and "System Settings" further enhances administrative control. The dashboard's overall goal is to provide a real-time, comprehensive overview of hospital operations, facilitating effective management and decision-making.

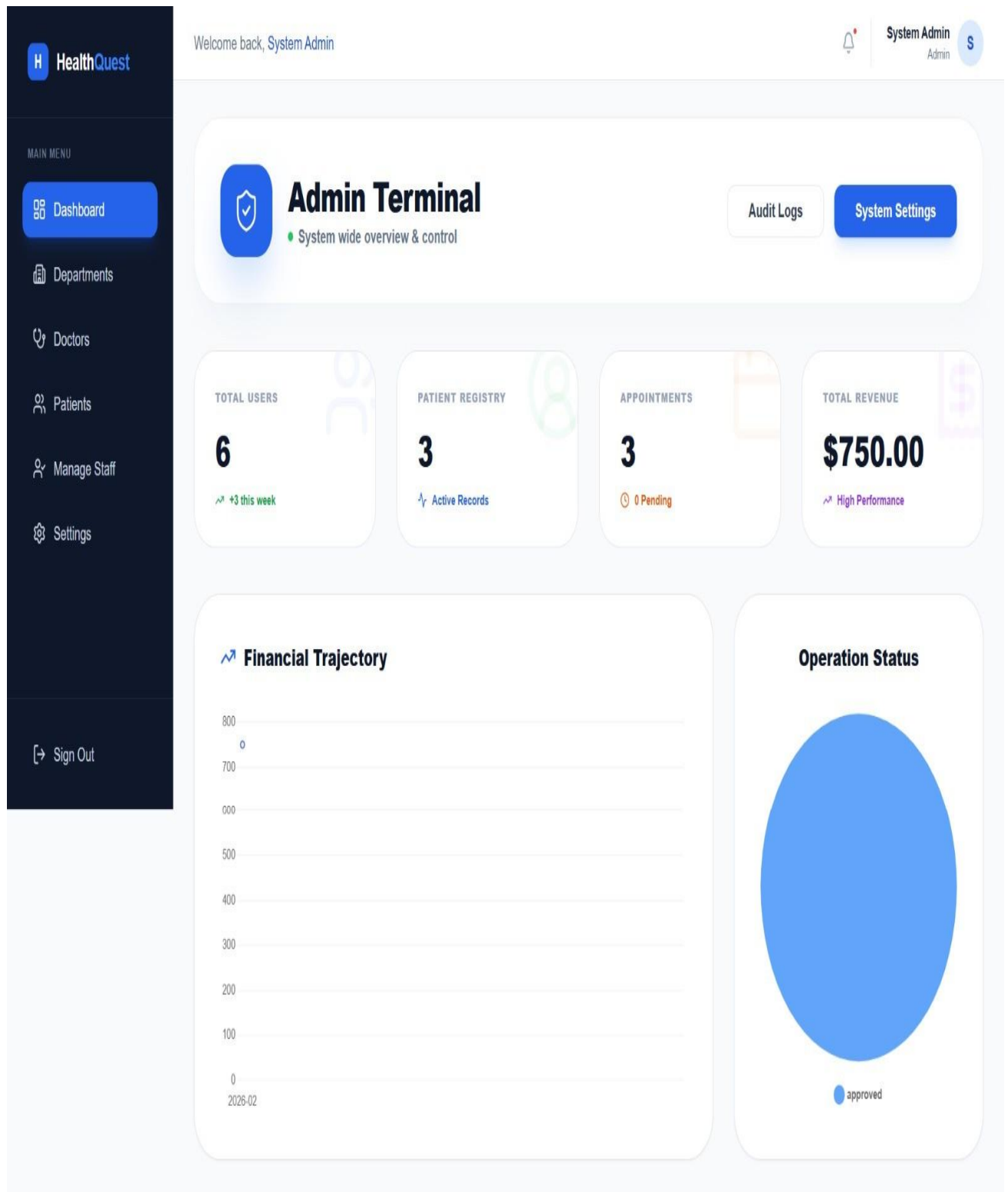


Fig:5.2 System Interface Description

Result:

The Enterprise Hospital Management System's implementation of the Admin Dashboard led to significant enhancements in system monitoring and administrative effectiveness. Administrators were able to monitor key performance indicators like user activity, patient records, appointment status, and revenue generation with the help of the dashboard, which successfully provided real-time insights into hospital operations. The system demonstrated the ability to process and display data dynamically, ensuring that administrators always have access to up-to-date information. With this real-time visibility, decision-making delays were reduced and overall operational control was improved.

The system's clear and simple interface also made it easier to use. Administrators did not need a lot of training to move between the various modules or carry out tasks. The structured layout of the dashboard ensured that important information was easily accessible, reducing the time required to analyze system data. Administrators were able to quickly identify trends and patterns as a result of the integration of visual elements like graphs and charts, which made it simpler to interpret complex data. For example, the financial trajectory graph helped in understanding revenue growth, while the operation status chart provided insights into system performance.

The implementation of the system also had a significant impact on improved data accuracy and centralized management. All hospital data was stored in a unified database, ensuring consistency and eliminating duplication. This data was efficiently retrieved and presented by the dashboard, giving a trustworthy overview of system operations. The integration of different modules ensured seamless communication between departments, improving coordination and workflow efficiency. For instance, updates in patient records or appointments were reflected instantly on the dashboard, ensuring transparency and accuracy.

Additionally, the system displayed robust reliability and security features. System settings and sensitive data could only be accessed by authorized administrators thanks to role-based access control. The inclusion of audit logs made it possible to keep track of system activities and be accountable. The system's continued safety and dependability for day-to-day operations were ensured by data handling and secure authentication procedures. Even when multiple users accessed the system simultaneously, the dashboard maintained stable performance, demonstrating its scalability and robustness. In conclusion, the Admin Dashboard of the Enterprise Hospital Management System proved to be an effective tool for managing hospital operations. Through real-time analytics, it supported better decision-making, increased efficiency, and increased data visibility. The combination of user-friendly design, accurate data management, and strong security measures makes the system suitable for real-world healthcare environments. The system has the potential to offer even more advantages in the future if it is further improved with features like cloud integration and advanced analytics.

3. Lab system interface

Description: The interface depicted is the Lab Diagnostics Module of the Enterprise Hospital Management System (HealthQuest), which was made to help laboratory technicians effectively manage diagnostic reports and patient test data. The layout follows a clean and structured dashboard design that enables quick interaction and easy navigation. On the left side, a vertical navigation panel is provided with the “Lab Tech” section highlighted, indicating that the current user role is a laboratory technician. As a result of this role-based interface, system security and usability are both enhanced. Users can only access features that are relevant to their roles. The top section of the interface displays a welcome message along with user identification, which helps in maintaining personalization and system awareness.

Report management and diagnostic procedures are the main topics covered in this section. The module's "Lab Diagnostics" heading is prominently displayed in the interface, indicating that it handles tasks related to pathology and report management. The Report Dispatch section, where the lab technician can select a patient from the registry and upload diagnostic reports in a secure PDF format, is one of the most important parts. The uploading of reports is made easier and more effective by the inclusion of a drag-and-drop upload area. By clicking the "Authorize & Sync Report" button, reports that have been uploaded are validated and synchronized with the system, making them immediately accessible to patients and doctors. The system's ability to automate and reduce manual labor is highlighted in this procedure. The interface has a status panel on the right side that shows the terminal's status as "Ready," which means that the system is working properly and is ready to process report uploads.

Additional indicators like "Secure Protocol" and "Live Sync" emphasize the system's real-time synchronization and data security. The inclusion of metrics such as uptime (99.9%) and encryption standard (AES-256) further highlights the system's reliability and security features. Users are reassured by these aspects that confidential medical information is handled safely and effectively. In general, the interface aims to simplify report management, reduce complexity, and streamline laboratory operations.

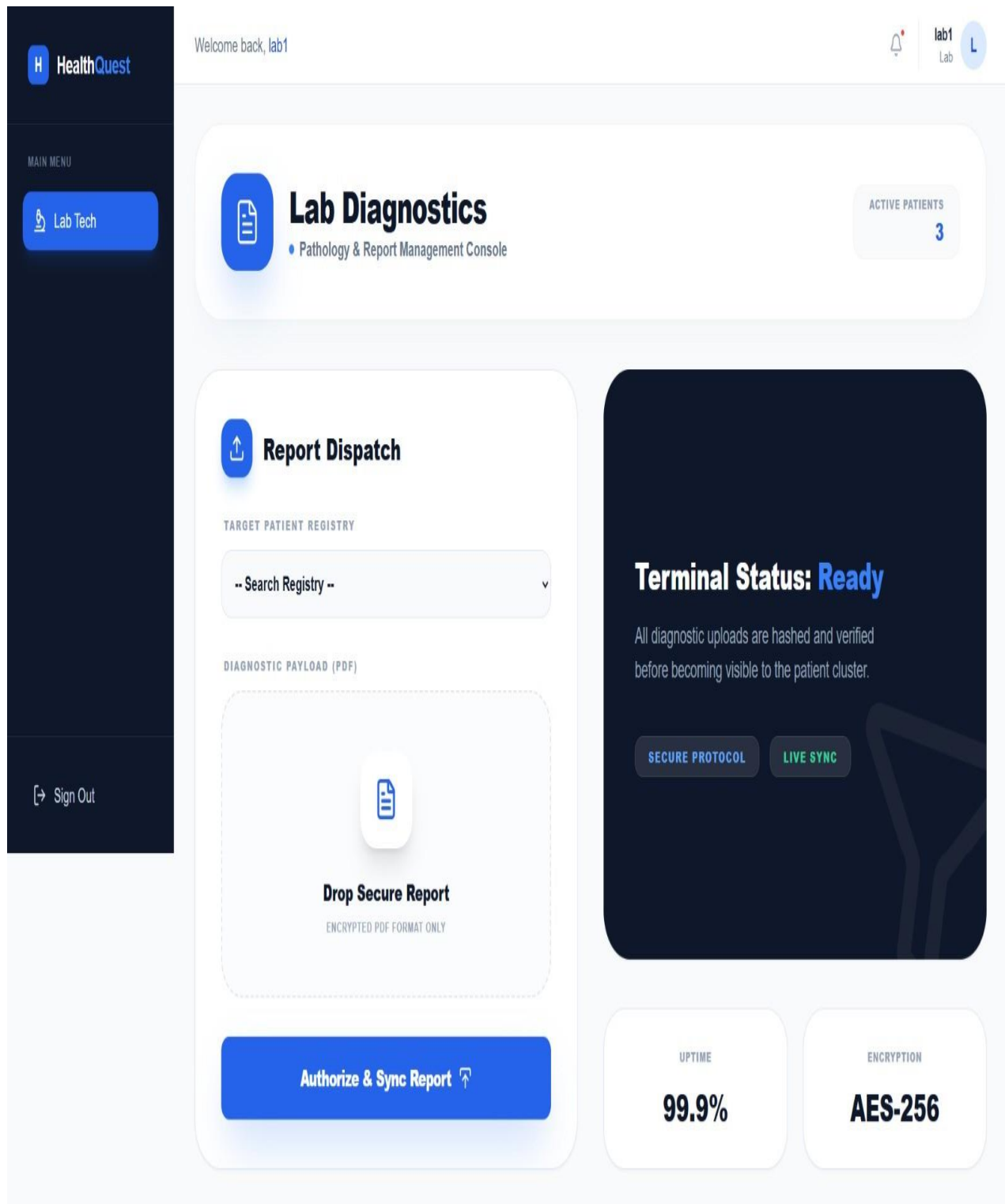


Fig:5.3 Lab system interface

Result:

The Enterprise Hospital Management System's Lab Diagnostics Module was implemented, which led to significant enhancements in laboratory efficiency and workflow. Laboratory technicians were able to upload, manage, and distribute digital diagnostic reports thanks to the system, which eliminated the need for paper forms. As a result of technicians being able to select patient records and upload reports directly into the system, the report dispatch process became faster and more accurate. This reduced delays in report delivery and improved the overall speed of healthcare services.

Real-time communication and data accessibility were also improved by the system. The system made a report immediately accessible to doctors and patients after it was uploaded and authorized. This eliminated the need for physical report collection and ensured that medical decisions could be made quickly based on accurate and up-to-date information. The live synchronization feature ensured that all system modules were updated simultaneously, improving coordination between departments such as laboratory, pharmacy, and general medicine.

An additional significant outcome was enhanced data reliability and security. Secure protocols and encryption standards like AES-256 were used to protect sensitive patient data while it was being transferred and stored. The high uptime of the system indicates stable performance and dependability. Role-based access control further ensured that only authorized users could upload or access lab reports, reducing the risk of data breaches.

Additionally, the user interface played a role in enhancing productivity and usability. Laboratory technicians were able to carry out their duties without requiring a lot of training thanks to the design's intuitive simplicity. The workflow was made more efficient by features like drag-and-drop file upload and clear status indicators. The system reduced manual errors and ensured consistency in report management.

Overall, the Lab Diagnostics Module was a useful part of the hospital management system that made laboratory operations more secure, accurate, and efficient. It contributed to better patient care by reducing diagnostic report turnaround times, enhancing departmental communication. The outcomes show that the system successfully achieves its goals and offers a dependable solution for contemporary healthcare environments.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

6.1 Conclusion

The **Enterprise Hospital Management System (HMS)** has been successfully designed and developed as a comprehensive solution to address the challenges faced in traditional hospital management practices. The system integrates multiple hospital operations such as patient registration, appointment scheduling, billing, pharmacy management, laboratory diagnostics, and administrative monitoring into a single unified platform. By leveraging modern web technologies and structured system design, the HMS provides an efficient, reliable, and scalable environment for managing healthcare services. The implementation of this system demonstrates how digital transformation can significantly improve the overall functioning of healthcare institutions.

One of the major achievements of the system is the **automation of manual processes**, which traditionally required significant time and effort. Tasks such as maintaining patient records, scheduling appointments, generating bills, and managing inventory were previously handled manually, leading to delays and errors. With the introduction of the HMS, these processes are automated and streamlined, reducing human intervention and improving accuracy. The system ensures that data is entered once and reused across multiple modules, eliminating redundancy and ensuring consistency. This not only saves time but also enhances the efficiency of hospital operations.

Another important aspect of the system is its **centralized data management**. All hospital-related information is stored in a structured database, allowing easy access and retrieval of data whenever required. Doctors can quickly access patient histories and lab reports, enabling better diagnosis and treatment.

decisions. Administrators can monitor hospital activities through dashboards and analytics, gaining insights into system performance and resource utilization. The centralized approach also improves coordination between different departments, ensuring that information flows seamlessly across the system.

The system also significantly enhances the **user experience** for all stakeholders, including patients, doctors, and administrators. The user-friendly interface, developed using modern frontend technologies, ensures that users can navigate the system بسهولة and perform tasks without difficulty. Patients can book appointments, view medical records, and download reports conveniently from any location. Doctors can access patient data in real time and update treatment information efficiently. Administrators can manage hospital operations through a centralized dashboard, improving decision-making and operational control. The responsive design ensures that the system is accessible across different devices, increasing flexibility and convenience.

Security and data protection are critical components of the system, and the HMS successfully implements **robust security mechanisms** to safeguard sensitive information. Features such as authentication, role-based access control, data encryption, and secure communication protocols ensure that only authorized users can access specific data. This is particularly important in healthcare systems where patient confidentiality must be maintained. The inclusion of audit logs and system monitoring further enhances accountability and transparency, ensuring that all system activities can be tracked and reviewed.

The system also demonstrates strong **performance and scalability**, making it suitable for real-world healthcare environments. The use of Node.js enables asynchronous processing, allowing the system to handle multiple user requests simultaneously without performance degradation. The MySQL database ensures efficient data storage and retrieval, even when handling large volumes of data.

The modular architecture of the system allows new features to be added easily, ensuring that the system can adapt to future requirements. This scalability makes the system suitable for both small clinics and large hospitals.

The **integration of different modules** within the system is another key strength. Modules such as patient management, appointment scheduling, billing, pharmacy, and laboratory diagnostics are interconnected, ensuring seamless data flow and communication. For example, when a doctor prescribes a test, the laboratory module receives the request instantly, and the billing system is updated automatically. This level of integration reduces delays, improves coordination, and enhances the overall efficiency of hospital operations. It also ensures that all stakeholders have access to accurate and up-to-date information.

The results obtained from the system implementation clearly indicate that the HMS improves **operational efficiency, data accuracy, and service quality**. The system reduces manual errors, enhances communication between departments, and provides real-time access to information. The use of dashboards and analytics tools enables administrators to monitor performance and make informed decisions. The system also reduces waiting times for patients and improves the overall healthcare experience. These benefits highlight the effectiveness of the system in addressing the limitations of traditional hospital management methods.

Despite its advantages, the system also has certain **limitations**, such as dependency on internet connectivity and the need for initial user training. However, these limitations are minor compared to the overall benefits provided by the system. With proper infrastructure and training, these challenges can be effectively managed. Future enhancements such as cloud deployment, mobile applications, and advanced analytics can further improve the system's capabilities and address existing limitations.

In conclusion, the Enterprise Hospital Management System represents a significant advancement in healthcare management by providing a **modern, efficient, and integrated solution**. It successfully achieves its objectives of automating hospital operations, improving data management, enhancing user experience, and ensuring data security. The system demonstrates strong performance, reliability, and scalability, making it suitable for real-world implementation. By adopting such systems, healthcare institutions can improve service quality, reduce operational costs, and provide better patient care.

Overall, the HMS not only simplifies hospital management but also contributes to the broader goal of digital transformation in the healthcare sector. It enables hospitals to move towards a more organized, data-driven, and patient-centric approach. With continuous improvements and technological advancements, the system has the potential to evolve further and play a crucial role in shaping the future of healthcare management.

6.2 Future Enhancement

Even though the Enterprise Hospital Management System (HMS) already offers a comprehensive and effective solution for managing hospital operations, there is a lot of room for growth and improvement. Integrating cutting-edge technologies, enhancing system scalability, enhancing user experience, and facilitating smarter healthcare decision-making are some possible future enhancements.

These enhancements will help transform the system into a more intelligent, flexible, and future-ready healthcare platform. The system's incorporation of Artificial Intelligence (AI) and Machine Learning (ML) will be a significant future improvement. By recognizing patterns in historical data, lab reports, and medical records, artificial intelligence

(AI) can be used to analyze patient data and assist physicians in diagnosis. Preventive care is made possible by predictive analytics, which can aid in the early detection of diseases and risk assessment. Additionally, routine tasks like appointment scheduling, chat-based patient support, and the summarization of medical reports can be automated with AI, easing the burden on healthcare professionals. Another important enhancement is the development of a mobile application for both patients and hospital staff. A mobile app can provide a more individualized and convenient experience than the current system, which can only be accessed through web browsers. Appointments, medication reminders, and lab results can all be sent directly to patients' mobile devices. Doctors and staff can access patient records and manage tasks on the go, improving efficiency and responsiveness.

Cloud-based deployment can also be implemented to improve the system. The system's scalability, availability, and data backup capabilities will all be enhanced upon moving to a cloud platform. The system can handle a lot of users and data at once thanks to its cloud integration. In addition, it supports remote healthcare services and multi-branch hospital management by ensuring that data is stored securely and accessible from any location. The incorporation of devices from the Internet of Things (IoT) is yet another useful improvement. Real-time patient data can be transmitted to the system by IoT-enabled medical devices like wearable health monitors, smart sensors, and connected diagnostic equipment. This allows continuous monitoring of patient health conditions and enables doctors to make timely decisions. For example, vital signs such as heart rate, blood pressure, and oxygen levels can be automatically recorded and analyzed.

Including features for telemedicine and video consultation can also enhance the system. This will make it possible for patients to talk to doctors without having to go to the hospital, which is especially helpful for patients who live in rural areas. Video consultation, online prescriptions, and digital health records can make healthcare services more accessible and convenient.

Another enhancement is the implementation of advanced analytics and reporting tools. The system can provide deeper insights into hospital operations, patient trends, and treatment outcomes by incorporating big data technologies. Service quality can be improved, operational costs can be reduced, and resource allocation can be optimized with these insights. In addition, accurate capacity management and patient inflow forecasting can be aided by predictive analytics. The system can also benefit from improved security and compliance features. Future versions can include biometric authentication, multi-factor authentication, and blockchain technology for secure data sharing. Medical records can be made more secure and trustworthy thanks to blockchain's ability to guarantee transparency and immutability. Another significant improvement is the integration with APIs and other healthcare systems. Insurance companies, pharmacy networks, and government health databases can all be connected to the system. This will enable seamless data exchange, faster insurance claim processing, and improved healthcare coordination.

Another area of improvement is the inclusion of a multilingual user interface, which will make the system accessible to users from different linguistic backgrounds. Because it ensures improved usability and inclusion, this is especially crucial in locations with diverse populations. Incorporating automated alert and notification systems can also improve the system. These alerts can notify doctors and staff about critical patient conditions, appointment reminders, and emergency situations. This will improve response time and ensure better patient care.

Natural language processing (NLP) and voice recognition both have the potential to further enhance user interaction. Doctors can use voice commands to update patient records, and patients can interact with the system using voice-based assistants. This reduces the need for manual data entry and enhances usability.

Finally, continuous improvement in user interface design and performance optimization will ensure that the system remains efficient and user-friendly. The system can be improved and adapted to changing healthcare requirements through regular updates, feedback collection, and usability testing. In conclusion, the Enterprise Hospital Management System's future enhancements will concentrate on making it more intelligent, accessible, secure, and scalable. By integrating advanced technologies such as AI, cloud computing, IoT, and telemedicine, the system can evolve into a next-generation healthcare platform. These enhancements will not only improve hospital operations but also enhance patient care, making healthcare services more efficient, personalized, and accessible.

APPENDIX

```
import { BrowserRouter as Router, Routes, Route, Navigate } from
'react-router-dom';
import { AuthProvider } from './context/AuthContext'; import
DashboardLayout from './layouts/DashboardLayout'; import Login
from './pages/Login';
import Register from './pages/Register';
import AdminDashboard from './pages/AdminDashboard'; import
DoctorDashboard from './pages/DoctorDashboard'; import
PatientDashboard from './pages/PatientDashboard'; import
Departments from './pages/Departments';

import Doctors from './pages/Doctors';
import Patients from './pages/Patients';
import Staff from './pages/Staff';
import BookAppointment from './pages/BookAppointment'; import
PharmacistDashboard from './pages/PharmacistDashboard'; import
LabDashboard from './pages/LabDashboard';
import ReceptionistDashboard from './pages/ReceptionistDashboard'; import
PatientBills from './pages/PatientBills';
import PatientReports from './pages/PatientReports'; import
LandingPage from './pages/LandingPage'; import {
ToastContainer } from 'react-toastify'; import 'react-
toastify/dist/ReactToastify.css';
```



```

const Placeholder = ({ title }) => (
  <div className="card">
    <h2 className="text-xl font-bold mb-4">{title}</h2>
    <p className="text-slate-500 italic text-sm">Module coming soon...</p>
  </div>
);

function App() { return (
  <AuthProvider>
    <Router>
      <Routes>
        <Route path="/" element={<LandingPage />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />

        <Route element={<DashboardLayout />>
          </> />
        </>
        { /* Admin Routes */}
        <Route path="admin" element={<AdminDashboard />} />
        <Route path="admin/departments" element={<Departments />} />
        <Route path="admin/doctors" element={<Doctors />} />
        <Route path="admin/patients" element={<Patients />} />
        <Route path="admin/staff" element={<Staff />} />
        <Route path="admin/settings" element={<Placeholder title="Settings"

          { /* Doctor Routes */}
          <Route path="doctor" element={<DoctorDashboard />} />
          <Route path="doctor/appointments" element={<DoctorDashboard />} />
          <Route path="doctor/patients" element={<Patients />} />

```

```

    { /* Patient Routes */
    <Route path="patient" element={<PatientDashboard />} />
    <Route path="patient/book" element={<BookAppointment />} />
    <Route path="patient/bills" element={<PatientBills />} />
    <Route path="patient/reports" element={<PatientReports />} />

    { /* Specialist Staff Routes */
    <Route path="receptionist" element={<ReceptionistDashboard />} />
    <Route path="pharmacist" element={<PharmacistDashboard />} />
    <Route path="lab" element={<LabDashboard />} />

    </Route>
  </Routes>

  <ToastContainer position="top-right" autoClose={3000} />
</Router>
</AuthProvider>
);
}

export default App;
@tailwind base; @tailwind
components; @tailwind
utilities;

@import url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700
&display=swap');

:root {
  --primary: 221.2 83.2% 53.3%;
  --primary-foreground: 210 40% 98%;
  --background: 210 40% 98%;
  --

```

```
    foreground: 222.2 84% 4.9%;
  --card: 0 0% 100%;
  --card-foreground: 222.2 84% 4.9%;
  --border: 214.3 31.8% 91.4%;
  --input: 214.3 31.8% 91.4%;
  --ring: 221.2 83.2% 53.3%;
  --radius: 0.75rem;
}

body {
  font-family: 'Outfit', sans-serif;
  @apply bg-slate-50 text-slate-900 antialiased;
}

@layer components {
  .card {
    @apply bg-white rounded-2xl border border-slate-200/60 shadow-sm transition-all
duration-300;
  }

  .card-hover {
    @apply hover:shadow-xl hover:shadow-blue-500/5 hover:-translate-y-1;
  }

  .btn-primary {
    @apply bg-blue-600 hover:bg-blue-700 text-white px-6 py-2.5 rounded-xl transition-all
duration-300 font-semibold shadow-lg shadow-blue-500/20 active:scale-95 disabled:opacity-
50;
  }
}
```

```

    .btn-secondary {
        @apply bg-white border border-slate-200 text-slate-700 hover:bg-slate-50 px-6 py-2.5
        rounded-xl transition-all duration-300 font-semibold active:scale-95;
    }

    .input-field {
        @apply w-full px-4 py-3 bg-white border border-slate-200 rounded-xl focus:outline-
        none focus:ring-4 focus:ring-blue-500/10 focus:border-blue-500 transition-all duration-300
        placeholder:text-slate-400;
    }

    .glass {
        @apply bg-white/70 backdrop-blur-md border border-white/20;
    }

    .badge {
        @apply px-3 py-1 rounded-full text-[10px] font-bold uppercase
        tracking-wider;
    }

    .animate-in {
        animation: slideIn 0.5s ease-out forwards;
    }

```

```
@keyframes slideIn {  
  from {  
    opacity: 0;  
    transform: translateY(10px);  
  }  
  
  to {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}
```

```
/* Custom Scrollbar */  
::-webkit-scrollbar {  
  width: 6px;  
}
```

```
::-webkit-scrollbar-track {  
  @apply bg-transparent;  
}
```

```
::-webkit-scrollbar-thumb {  
  @apply bg-slate-200 rounded-full hover:bg-slate-300;  
}
```

```
// Enterprise Hospital Management System - Core Server const  
express = require('express');  
const cors = require('cors')
```

```
const morgan = require('morgan'); const
path = require('path');
require('dotenv').config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());
app.use(morgan('dev'));
app.use('/uploads', express.static(path.join(____dirname, 'uploads')));

// Routes (to be implemented)
app.use('/api/auth', require('./routes/authRoutes')); app.use('/api/users',
require('./routes/userRoutes')); app.use('/api/departments',
require('./routes/departmentRoutes')); app.use('/api/doctors',
require('./routes/doctorRoutes')); app.use('/api/patients',
require('./routes/patientRoutes')); app.use('/api/appointments',
require('./routes/appointmentRoutes')); app.use('/api/billing',
require('./routes/billingRoutes')); app.use('/api/pharmacy',
require('./routes/pharmacyRoutes')); app.use('/api/lab',
require('./routes/labRoutes')); app.use('/api/prescriptions',
require('./routes/prescriptionRoutes')); app.use('/api/dashboard',
require('./routes/dashboardRoutes'));

// Global Error Handler app.use((err, req,
res, next) => {
```

```

    console.error(err.stack);
    res.status(err.status || 500).json({
      success: false,
      message: err.message || 'Internal Server Error'
    });
  });
});

const PORT = process.env.PORT || 5000; app.listen(PORT, ()
=> {
  console.log(`Server running in ${process.env.NODE_ENV} mode on port
${PORT}`);
});

const pool = require('./config/db');

const fixData = async () => { try {
  console.log('Starting DB fix...');

  // 1. Find doctors without records const
  [doctors] = await pool.query(
    "SELECT u.id, u.name FROM users u LEFT JOIN doctors d ON u.id = d.user_id
WHERE u.role = 'doctor' AND d.id IS NULL"
  );

  for (const u of doctors) {
    console.log("Adding doctor record for: " + u.name); await
    pool.query(
      "INSERT INTO doctors (user_id, specialization, experience) VALUES(?, ?, ?)",
      );
  }
}

```

```
[u.id, "General Physician", 5]
```

```
    // 2. Find patients without records const
    [patients] = await pool.query(
        "SELECT u.id, u.name FROM users u LEFT JOIN patients p ON u.id = p.user_id
WHERE u.role = 'patient' AND p.id IS NULL"
    );
    for (const u of patients) {
        console.log("Adding patient record for: " + u.name);
        await pool.query("INSERT INTO patients (user_id) VALUES (?)",
[u.id]);
    }
```

```
        console.log('DB fix completed.');
```

```
    process.exit(0);
} catch (err) {
    console.error('Fix failed:', err);
    process.exit(1);
}
};
```

```
fixData();
```


Admin Dashboard. Jsx

```
import { useEffect, useState } from 'react';
import api from '../services/api';
import {
  Users, UserRound, DollarSign, Calendar, TrendingUp, ShieldCheck, Clock, Receipt,
  UserCircle2, Activity
} from 'lucide-react';
import {
  Chart as ChartJS, CategoryScale, LinearScale, PointElement,
  LineElement, Title, Tooltip, Legend, ArcElement
} from 'chart.js';
import { Line, Pie } from 'react-chartjs-2';
ChartJS.register(
  CategoryScale, LinearScale, PointElement, LineElement,
  Title, Tooltip, Legend, ArcElement
);

const StatsCard = ({ title, value, icon: Icon, color }) => (
  <div className="card flex items-center gap-6">
    <div className={`p-4 rounded-2xl ${color}`}>
      <Icon size={28} />
    </div>
    <div>
      <p className="text-slate-500 text-sm font-medium">{title}</p>
      <h3 className="text-2xl font-bold text-slate-800 mt-1">{value}</h3>
    </div>
  </div>
);
```

```
const AdminDashboard = () => {
  const [stats, setStats] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchStats();
  }, []);

  const fetchStats = async () => {
    try {
      const res = await api.get('/dashboard/stats');
      setStats(res.data.stats);

    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  if (loading) return <div>Loading...</div>;

  const lineData = {
    labels: stats?.monthly_revenue?.map(m => m.month).reverse() || [],
    datasets: [{
      label: 'Revenue ($)',
      data: stats?.monthly_revenue?.map(m => m.total).reverse() || [],
      borderColor: '#1e40af',
      backgroundColor: 'rgba(30, 64, 175, 0.1)',
    }],
  };
}
```

```

    fill: true,
    tension: 0.4
  ]
};

```

```

const pieData = {
  labels: stats?.appointment_stats?.map(s => s.status) || [],
  datasets: [{
    data: stats?.appointment_stats?.map(s => s.count) || [],
    backgroundColor: ['#60a5fa', '#10b981', '#f59e0b', '#ef4444'],
    borderWidth: 0
  }]
};

```

```

return (

```

```

<div className="space-y-10 animate-in">
  <header className="flex flex-col md:flex-row justify-between items-start md:items-center gap-8 bg-white p-10 rounded-[2.5rem] border border-slate-200/60 shadow-xl shadow-slate-200/10">
    <div className="flex items-center gap-6">
      <div className="w-20 h-20 bg-blue-600 text-white rounded-3xl flex items-center justify-center shadow-2xl shadow-blue-500/40">
        <ShieldCheck size={40} strokeWidth={1.5} />
      </div>
      <div>
        <h1 className="text-4xl font-black text-slate-900 tracking-tight">Admin
Terminal</h1>

```

```

        <p className="text-slate-500 mt-1 font-semibold flex items-center gap-2">
            <span className="w-2 h-2 bg-green-500 rounded-full animate-
pulse"></span>
                System wide overview & control
            </p>
        </div>
    </div>
    <div className="flex gap-4">
        <button className="btn-secondary px-8 font-bold">Audit Logs</button>
        <button className="btn-primary px-8">System Settings</button>
    </div>
</header>

<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-8">
    <div className="card-hover bg-white p-8 rounded-[2rem] border border-slate-200
overflow-hidden relative group">
        <Users className="absolute -right-4 -top-4 w-24 h-24 text-blue-500/5 group-
hover:scale-110 transition-transform" />

        <p className="text-[11px] font-black text-slate-400 uppercase tracking-widest mb-
4">Total Users</p>
        <p className="text-4xl font-black text-slate-900">{stats?.total_patients +
stats?.total_doctors || 0}</p>
        <div className="mt-4 flex items-center gap-2 text-xs font-bold text-green-600">
            <TrendingUp size={14} /> +3 this week
        </div>
    </div>
    <div className="card-hover bg-white p-8 rounded-[2rem] border border-slate-200
overflow-hidden relative group"

```

```

        <UserCircle2 className="absolute -right-4 -top-4 w-24 h-24 text-green-500/5
group-hover:scale-110 transition-transform" />
        <p className="text-[11px] font-black text-slate-400 uppercase tracking-widest mb-
4">Patient Registry</p>
        <p className="text-4xl font-black text-slate-900">{stats?.total_patients}</p>
        <div className="mt-4 flex items-center gap-2 text-xs font-bold text-blue-600">
            <Activity size={14} /> {stats?.total_patients > 0 ? 'Active Records' : 'No Data'}
        </div>
    </div>
    <div className="card-hover bg-white p-8 rounded-[2rem] border border-slate-200
overflow-hidden relative group">
        <Calendar className="absolute -right-4 -top-4 w-24 h-24 text-orange-500/5 group-
hover:scale-110 transition-transform" />
        <p className="text-[11px] font-black text-slate-400 uppercase tracking-widest mb-
4">Appointments</p>
        <p className="text-4xl font-black text-slate-
900">{stats?.appointment_stats?.reduce((acc, curr) => acc + curr.count, 0) || 0}</p>
        <div className="mt-4 flex items-center gap-2 text-xs font-bold text-orange-600">
            <Clock size={14} /> {stats?.appointment_stats?.find(s => s.status ===
'pending')?.count || 0} Pending
        </div>
    </div>
    <div className="card-hover bg-white p-8 rounded-[2rem] border border-slate-200
overflow-hidden relative group">
        <Receipt className="absolute -right-4 -top-4 w-24 h-24 text-purple-500/5 group-
hover:scale-110 transition-transform" />
        <p className="text-[11px] font-black text-slate-400 uppercase tracking-widest mb-
4">Total Revenue</p>

```

```

        <p className="text-4xl font-black text-slate-
900">${stats?.total_revenue?.toLocaleString() || '0'}</p>
        <div className="mt-4 flex items-center gap-2 text-xs font-bold text-purple-600">
            <TrendingUp size={14} /> High Performance
        </div>
    </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
    <div className="lg:col-span-2 card bg-white p-10 rounded-[2.5rem] border border-
slate-200 shadow-sm">
        <h3 className="text-xl font-black text-slate-900 mb-8 flex items-center gap-3">
            <TrendingUp size={24} className="text-blue-600" />
            Financial Trajectory
        </h3>
        <div className="h-[350px]">
            <Line data={lineData} options={{
                maintainAspectRatio: false,
                plugins: { legend: { display: false } },
                scales: {
                    y: { beginAtZero: true, grid: { color: '#f1f5f9' }, border: { display: false } },
                    x: { grid: { display: false }, border: { display: false } }
                }
            }} />
        </div>
    </div>
    <div className="card bg-white p-10 rounded-[2.5rem] border border-slate-200
shadow-sm">

```

```

    <h3 className="text-xl font-black text-slate-900 mb-8 text-center">Operation
Status</h3>
    <div className="h-[350px] flex items-center justify-center">
      <Pie data={pieData} options={{
        maintainAspectRatio: false,
        plugins: { legend: { position: 'bottom', labels: { usePointStyle: true, padding:
20, font: { weight: 'bold' } } } }
      }} />
    </div>
  </div>
</div>
</div>
);
};

```

export default AdminDashboard;

Book Appointment.jsx

```

import { useEffect, useState } from 'react';
import api from '../services/api';
import { toast } from 'react-toastify';
import {
  Calendar,
  User,
  Clock,
  Check,

```

```
Stethoscope,  
Search,  
ChevronRight,  
Building2,  
ShieldCheck  
} from 'lucide-react';
```

```
const BookAppointment = () => {  
  const [doctors, setDoctors] = useState([]);  
  const [departments, setDepartments] = useState([]);  
  const [formData, setFormData] = useState({ doctor_id: "", appointment_date: " });  
  const [patientData, setPatientData] = useState(null);  
  const [selectedDept, setSelectedDept] = useState(null);  
  const [searchTerm, setSearchTerm] = useState("");
```

```
  useEffect(() => {  
    fetchInitialData();  
  }, []);
```

```
  const fetchInitialData = async () => {  
    try {  
      const [docsRes, deptsRes, profileRes] = await Promise.all([  
        api.get('/doctors'),  
        api.get('/departments'),  
        api.get('/patients/profile/me')  
      ]);  
      setDoctors(docsRes.data.doctors);  
      setDepartments(deptsRes.data.departments);  
      setPatientData(profileRes.data.patient);
```



```

// Auto-select first department if available

if (deptsRes.data.departments.length > 0) {
  setSelectedDept(deptsRes.data.departments[0].id);
}
} catch (err) {
  console.error(err);
  toast.error('Failed to load portal data');
}
};

const handleSubmit = async (e) => {
  e.preventDefault();
  if (!formData.doctor_id) return toast.warning('Please select a doctor');
  if (!formData.appointment_date) return toast.warning('Please select a date and time');

  try {
    await api.post('/appointments', formData);
    toast.success('Appointment requested successfully!');
    setFormData({ doctor_id: '', appointment_date: '' });
  } catch (err) {
    toast.error(err.response?.data?.message || 'Booking failed');
  }
};

const filteredDoctors = doctors.filter(doc => {
  const matchesDept = selectedDept ? doc.department_id === selectedDept : true

```

```

const matchesSearch = doc.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
  doc.specialization.toLowerCase().includes(searchTerm.toLowerCase());
return matchesDept && matchesSearch;
});

```

```

return (

```

```

    <div className="max-w-6xl mx-auto space-y-8 pb-12">
    <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
      <div>
        <h1 className="text-3xl font-bold text-slate-900">Schedule Consultation</h1>
        <p className="text-slate-500 mt-1">Select a specialist and choose your preferred
time</p>
      </div>
      {patientData && (
        <div className="bg-blue-50 px-4 py-2 rounded-2xl border border-blue-100 flex
items-center gap-3">
          <div className="w-8 h-8 bg-blue-600 rounded-lg flex items-center justify-center
text-white font-bold">
            {patientData.name[0]}
          </div>
          <div>
            <p className="text-xs font-bold text-blue-800 tracking-wide
uppercase">Patient ID</p>
            <p className="text-sm font-bold text-blue-600">#P-{patientData.id}</p>
          </div>
        </div>
      )}
    </div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12 gap-8">
  {/* Left Sidebar: Departments */}
  <div className="lg:col-span-3 space-y-4">
    <h3 className="text-xs font-bold text-slate-400 uppercase tracking-widest px-2">Clinical Departments</h3>
    <div className="flex lg:flex-col gap-2 overflow-x-auto pb-4 lg:pb-0 no-scrollbar">
      {departments.map(dept => (
        <button
          key={dept.id}
          onClick={() => setSelectedDept(dept.id)}
          className={`flex items-center justify-between px-4 py-3 rounded-xl
            transition-all whitespace-nowrap lg:whitespace-normal text-left ${selectedDept === dept.id
              ? 'bg-blue-600 text-white shadow-lg shadow-blue-600/20 font-bold'
              : 'bg-white border border-slate-100 text-slate-600 hover:bg-slate-50'
            }`}
          >
            <span className="flex items-center gap-2">
              <Building2 size={16} />
              {dept.name}
            </span>
            <ChevronRight size={14} className={selectedDept === dept.id ? 'opacity-100' : 'opacity-0'} />
          </button>
        )]}
    </div>
  </div>

  {/* Main Content: Doctor Selection & Booking */}
  <div className="lg:col-span-9 space-y-6">

```

```

<div className="card border-none shadow-sm flex flex-col md:flex-row gap-4
justify-between items-center p-4">
  <div className="relative w-full md:w-96">
    <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400"
size={18} />
    <input
      className="input-field pl-10 h-11"
      placeholder="Search by name or specialty..."
      value={searchTerm}
      onChange={e => setSearchTerm(e.target.value)}
    />
  </div>
  <div className="text-xs font-bold text-slate-400 uppercase tracking-widest">
    Showing {filteredDoctors.length} Specialists
  </div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
  {filteredDoctors.map(doc => (
    <div
      key={doc.id}
      onClick={() => setFormData({ ...formData, doctor_id: doc.id })}
      className={`card group cursor-pointer transition-all border-2 relative
overflow-hidden ${formData.doctor_id === doc.id
        ? 'border-blue-600 bg-blue-50/50 ring-4 ring-blue-600/5'
        : 'border-slate-100 hover:border-blue-300'
      }`}
    )
  )}

```

```

>
<div className="flex items-start gap-4">
  <div className="w-16 h-16 bg-white rounded-2xl flex items-center
justify-center shadow-sm border border-slate-50 text-blue-700 font-bold text-xl ring-4 ring-slate-
50">

    {doc.name[0]}
  </div>
  <div className="flex-1">
    <div className="flex items-center gap-2">
      <h4 className="font-bold text-slate-800 text-lg">Dr.
{doc.name}</h4>
      {doc.availability_status && <ShieldCheck size={16}
className="text-emerald-500" />}
    </div>

    <p className="text-sm text-blue-600 font-bold mb-
1">{doc.specialization}</p>
    <p className="text-xs text-slate-500 font-medium">{doc.experience}
Years Experience</p>
  </div>
  {formData.doctor_id === doc.id && (
    <div className="absolute top-4 right-4 text-blue-700">
      <div className="w-6 h-6 bg-blue-600 text-white rounded-full flex
items-center justify-center">
        <Check size={14} strokeWidth={4} />
      </div>
    </div>
  )}
</div>
</div>

```

```
    )}
  </div>
```

```
{filteredDoctors.length === 0 && (
  <div className="card py-16 text-center text-slate-400 italic">
    No specialists found matching your search.
  </div>
)}
```

```
{formData.doctor_id && (
  <div className="card bg-slate-900 border-none text-white p-8 space-y-6
shadow-2xl animate-in fade-in slide-in-from-bottom-4">
    <h3 className="text-xl font-bold flex items-center gap-2">
      <Clock className="text-blue-500" /> Finalize Appointment
    </h3>
    <form onSubmit={handleSubmit} className="flex flex-col md:flex-row gap-6
items-end">
      <div className="flex-1 w-full">
        <label className="block text-[10px] uppercase font-bold tracking-widest
text-slate-400 mb-2 px-1">Choose Date & Time</label>
        <div className="relative">
          <Calendar className="absolute left-4 top-1/2 -translate-y-1/2 text-
blue-500" size={20} />
          <input
            type="datetime-local"
            className="w-full h-14 bg-slate-800 border-none rounded-2xl pl-12
pr-4 text-lg font-bold focus:ring-2 focus:ring-blue-500 text-white"
            required
            value={formData.appointment_date}
            onChange={e => setFormData({ ...formData, appointment_date:
```

```

e.target.value }}}
        />
      </div>
    </div>
    <button type="submit" className="h-14 bg-blue-600 hover:bg-blue-700
text-white font-bold px-8 rounded-2xl transition-all flex items-center gap-2 group whitespace-
nowrap">
      Request Consultation <ChevronRight size={20} className="group-
hover:translate-x-1 transition-transform" />
    </button>
  </form>
</div>
)}
</div>
</div>
</div>
);
};

```

Departments.jsx

```

import { useEffect, useState } from 'react';
import api from '../services/api';
import { toast } from 'react-toastify';
import { Building2, Plus, Trash2, Edit } from 'lucide-react';

const Departments = () => {
  const [departments, setDepartments] = useState([]);
  const [showModal, setShowModal] = useState(false);
  const [formData, setFormData] = useState({ name: "", description: "" });

```

```
useEffect(() => {  
  fetchDepartments();  
}, []);
```

```
const fetchDepartments = async () => {  
  try {  
    const res = await api.get('/departments');  
    setDepartments(res.data.departments);  
  } catch (err) {  
    toast.error('Failed to load departments');  
  }  
};
```

```
const handleSubmit = async (e) => {  
  e.preventDefault();  
  try {  
    await api.post('/departments', formData);  
    toast.success('Department added');
```

```
setShowModal(false);  
    setFormData({ name: "", description: "" });  
    fetchDepartments();  
  } catch (err) {  
    toast.error('Failed to add department');  
  }  
};
```

```
const deleteDept = async (id) => {  
  if (window.confirm('Are you sure you want to delete this department?')) {  
    try {
```



```

    await api.delete(`/departments/${id}`);
    toast.success('Department deleted');
    fetchDepartments();
  } catch (err) {
    toast.error('Failed to delete department');
  }
}
};

```

```

return (

```

```

  <div className="space-y-10 animate-in">

```

```

    <header className="flex flex-col md:flex-row justify-between items-start md:items-center gap-8 bg-white p-10 rounded-[2.5rem] border border-slate-200/60 shadow-xl shadow-slate-200/10">

```

```

      <div className="flex items-center gap-6">

```

```

        <div className="w-20 h-20 bg-blue-600 text-white rounded-3xl flex items-center justify-center shadow-2xl shadow-blue-500/40">

```

```

          <Building2 size={40} strokeWidth={1.5} />

```

```

        </div>

```

```

      <div>

```

```

<h1 className="text-4xl font-black text-slate-900 tracking-tight">Medical Units</h1>

```

```

  <p className="text-slate-500 mt-1 font-semibold flex items-center gap-2">

```

```

    <span className="w-2 h-2 bg-blue-500 rounded-full animate-pulse"></span>

```

```

    Specialized Care & Clinical Divisions

```

```

  </p>

```

```

</div>

```

```

</div>

```

```

<button

```

```

  onClick={() => setShowModal(true)}

```

```

      className="btn-primary group px-8"
    >
      <Plus size={20} className="transition-transform group-hover:scale-110" />
      New Department
    </button>
  </header>

  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
    {departments.map((dept) => (
      <div key={dept.id} className="card-hover group bg-white rounded-[2.5rem]
border border-slate-200/60 shadow-lg shadow-slate-200/20 overflow-hidden flex flex-col h-
full">
        <div className="p-10 flex-1">
          <div className="flex justify-between items-start mb-8">
            <div className="p-5 bg-gradient-to-br from-blue-50 to-blue-100 text-blue-
600 rounded-[1.5rem] shadow-sm group-hover:scale-110 transition-transform">
              <Building2 size={32} />
            </div>
            <button
              onClick={() => deleteDept(dept.id)}

              className="w-10 h-10 bg-white shadow-xl border border-slate-100 text-red-600 rounded-2xl
flex items-center justify-center opacity-0 group-hover:opacity-100 transition-all hover:scale-
110"
            >
              <Trash2 size={18} />
            </button>
          </div>
          <h3 className="text-2xl font-black text-slate-900 mb-3 uppercase tracking-
tight">{dept.name}</h3>

```

```

        <p className="text-slate-500 text-sm font-medium leading-
relaxed">{dept.description}</p>
    </div>
    <div className="px-10 py-6 bg-slate-50/50 border-t border-slate-100 flex items-
center justify-between">
        <span className="text-[10px] font-black text-blue-600 uppercase tracking-
widest">Active Division</span>
        <div className="flex items-center gap-2">
            <span className="w-2 h-2 bg-green-500 rounded-full"></span>
            <span className="text-[10px] font-bold text-slate-400 uppercase tracking-
widest">Operational</span>
        </div>
    </div>
</div>
)}}
</div>

```

```

{showModal && (
    <div className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items-center
justify-center z-50 p-4">
        <div className="bg-white rounded-2xl w-full max-w-md p-8 shadow-2xl">
            <h2 className="text-xl font-bold text-slate-800 mb-6">New Department</h2>

<form onSubmit={handleSubmit} className="space-y-4">
    <div>
        <label className="block text-sm font-medium text-slate-700 mb-
1">Name</label>
        <input
            className="input-field"

```

```

        required
        value={formData.name}
        onChange={(e) => setFormData({ ...formData, name: e.target.value })}
      />
    </div>
    <div>
      <label className="block text-sm font-medium text-slate-700 mb-
1">Description</label>
      <textarea
        className="input-field min-h-[100px]"
        required
        value={formData.description}
        onChange={(e) => setFormData({ ...formData, description: e.target.value
    }}}

      ></textarea>
    </div>
    <div className="flex gap-4 pt-4">
      <button
        type="button"
        onClick={() => setShowModal(false)}
        className="flex-1 px-4 py-2 border border-slate-200 rounded-lg text-
slate-600 hover:bg-slate-50 font-medium transition-colors"
        >
        Cancel
      </button>
      <button type="submit" className="flex-1 btn-primary">
        Create
      </button>
    </div>

```

```
        </form>
      </div>
    </div>
  )}
</div>
);
};
```

```
export default Departments;
```

Patientreports.jsx

```
import { useEffect, useState } from 'react';
import api from '../services/api';
import { toast } from 'react-toastify';
import { FileText, Download, Calendar, Search } from 'lucide-react';
```

```
const PatientReports = () => {
  const [reports, setReports] = useState([]);
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
    fetchMyReports();
  }, []);
```

```
  const fetchMyReports = async () => {
```

```
    try {
      const profileRes = await api.get('/patients/profile/me');
      const me = profileRes.data.patient;
```

```

    if (me) {
      const res = await api.get(`/lab/patient/${me.id}`);
      setReports(res.data.reports);
    }
  } catch (err) { toast.error('Failed to load reports'); }
  finally { setLoading(false); }
};

```

```

return (
  <div className="space-y-10 animate-in pb-12">
    <header className="flex flex-col md:flex-row justify-between items-start md:items-center gap-8 bg-white p-10 rounded-[2.5rem] border border-slate-200/60 shadow-xl shadow-slate-200/10">
      <div className="flex items-center gap-6">
        <div className="w-20 h-20 bg-purple-600 text-white rounded-3xl flex items-center justify-center shadow-2xl shadow-purple-500/40">
          <FileText size={40} strokeWidth={1.5} />
        </div>
        <div>
          <h1 className="text-4xl font-black text-slate-900 tracking-tight">Diagnostics
Vault</h1>
          <p className="text-slate-500 mt-1 font-semibold flex items-center gap-2">
            <span className="w-2 h-2 bg-purple-500 rounded-full animate-pulse"></span>
            Secure clinical results and laboratory records
          </p>
        </div>
      </div>
    </header>
  </div>
);

```

</header>

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-8">
  {reports.map(report => (
    <div key={report.id} className="card-hover group bg-white rounded-[2rem]
border border-slate-200/60 shadow-lg shadow-slate-200/20 overflow-hidden relative">
      <div className="p-8 pb-4">
        <div className="flex justify-between items-start mb-8">
          <div className="w-16 h-16 bg-purple-100 text-purple-600 rounded-2xl flex
items-center justify-center group-hover:bg-purple-600 group-hover:text-white transition-all
duration-500 shadow-sm border border-purple-100">
            <FileText size={28} />
          </div>
          <p className="text-[10px] font-black text-slate-300 uppercase tracking-
widest">ID-RPT-{report.id}</p>
        </div>
        <h3 className="font-black text-xl text-slate-900 mb-2">Diagnostic
Outcome</h3>
        <div className="flex items-center gap-2 text-[10px] font-bold text-slate-400
uppercase tracking-widest mb-6">
          <Calendar size={14} className="text-purple-400" />
          {new Date(report.uploaded_at).toLocaleDateString(undefined, {
            month: 'short', day: 'numeric', year: 'numeric'
          })}
        </div>
      </div>
    </div>
  )}
```

```

<a
    href={`http://localhost:5000/${report.file_path}`}
    target="_blank"
    rel="noreferrer"
    className="bg-slate-900 hover:bg-black text-white w-full py-4 rounded-2xl
font-black text-sm transition-all flex items-center justify-center gap-3 shadow-xl shadow-slate-
200 group/btn"
    >
    <Download size={18} className="group-hover/btn:translate-y-1 transition-
transform" />
    Secure Access
  </a>
</div>
</div>
)}}
{reports.length === 0 && !loading && (
  <div className="col-span-full card border-dashed py-16 flex flex-col items-center
justify-center text-slate-400">
    <Search size={48} className="mb-4 opacity-20" />
    <p>No medical reports found in your records.</p>
  </div>
)}
</div>
</div>
);
};

export default PatientReports;

```


Patients.jsx

```
import { useEffect, useState } from 'react';
import api from '../services/api';
import { toast } from 'react-toastify';
import { Users, Trash2, Phone, MapPin, Droplet } from 'lucide-react';

const Patients = () => {
  const [patients, setPatients] = useState([]);

  useEffect(() => { fetchPatients(); }, []);

  const fetchPatients = async () => {
    try {
      const res = await api.get('/patients');
      setPatients(res.data.patients);
    } catch (err) { toast.error('Failed to load patients'); }
  };

  const deletePatient = async (id) => {
    if (window.confirm('Delete this patient and their user account?')) {
      try {
        await api.delete(`/users/${id}`); // We delete the user which cascades to patient
        toast.success('Patient deleted');
        fetchPatients();
      } catch (err) { toast.error('Delete failed'); }
    }
  };

  return (
    <div className="space-y-10 animate-in">
```

```

<header className="flex flex-col md:flex-row justify-between items-start md:items-center gap-8 bg-white p-10 rounded-[2.5rem] border border-slate-200/60 shadow-xl shadow-slate-200/10">
  <div className="flex items-center gap-6">
    <div className="w-20 h-20 bg-blue-600 text-white rounded-3xl flex items-center justify-center shadow-2xl shadow-blue-500/40">
      <Users size={40} strokeWidth={1.5} />
    </div>
    <div>
      <h1 className="text-4xl font-black text-slate-900 tracking-tight">Patient
Directory</h1>
      <p className="text-slate-500 mt-1 font-semibold flex items-center gap-2">
        <span className="w-2 h-2 bg-blue-500 rounded-full animate-pulse"></span>
        Centralized Health Record Registry
      </p>
    </div>
  </div>
  <div className="flex bg-slate-100 p-1.5 rounded-2xl border border-slate-200 shadow-inner">
    <div className="px-6 py-2 bg-white rounded-xl shadow-sm text-xs font-black text-blue-600 uppercase tracking-widest">
      Total: {patients.length}
    </div>
  </div>
</header>

<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-8">
  {patients.map(p => (
    <div key={p.id} className="card-hover group bg-white rounded-[2rem] border border-slate

```

```

200/60 shadow-lg shadow-slate-200/20 overflow-hidden relative">
    <div className="absolute top-0 right-0 p-4 opacity-0 group-hover:opacity-100
transition-opacity">
        <button
            onClick={() => deletePatient(p.user_id)}
            className="w-10 h-10 bg-white shadow-xl border border-slate-100 text-
red-600 rounded-2xl flex items-center justify-center hover:scale-110 active:scale-95 transition-
all"
        >
            <Trash2 size={18} />
        </button>
    </div>

    <div className="p-8 pb-4">
        <div className="w-16 h-16 bg-gradient-to-br from-blue-500 to-blue-700 text-
white rounded-[1.5rem] flex items-center justify-center font-black text-2xl shadow-xl shadow-
blue-200 mb-6 group-hover:rotate-6 transition-transform">
            {p.name[0]}
        </div>
        <h3 className="font-black text-xl text-slate-900 leading-tight mb-
1">{p.name}</h3>
        <p className="text-[10px] font-black text-slate-400 uppercase tracking-
[0.2em]">{p.email}</p>
    </div>

    <div className="p-8 pt-0 space-y-4">
        <div className="grid grid-cols-2 gap-3">
            <div className="bg-slate-50 rounded-2xl p-4 text-center border border-
slate-100">

```

```

<p className="text-[10px] font-bold text-slate-400 uppercase mb-1">Blood</p>
    <p className="text-lg font-black text-red-600 flex items-center justify-
center gap-1">

        <Droplet size={14} strokeWidth={4} />
        {p.blood_group || 'N/A'}
    </p>
</div>

<div className="bg-slate-50 rounded-2xl p-4 text-center border border-
slate-100 flex items-center justify-center">
    <div className="text-center">
        <p className="text-[10px] font-bold text-slate-400 uppercase mb-
1">Status</p>

        <span className="w-3 h-3 bg-emerald-500 rounded-full block mx-
auto ring-4 ring-emerald-50 shadow-sm shadow-emerald-200"></span>
    </div>
</div>
</div>

<div className="bg-blue-50/50 rounded-2xl p-4 flex items-center gap-3
border border-blue-100/50">
    <Phone size={16} className="text-blue-600" />
    <span className="text-sm font-bold text-blue-900">{p.phone || 'No
contact'}</span>
</div>

<div className="flex items-center gap-3 px-1">
    <MapPin size={14} className="text-slate-400 shrink-0" />
    <span className="text-[11px] font-medium text-slate-500 line-clamp-

```

```

1">{p.address || 'Address unverified'}</span>
      </div>
    </div>
  </div>
  )}
</div>
</div>
);
};

```

```
export default Patients;
```

PharmacistDashboard.jsx

```

import { useEffect, useState } from 'react';
import api from '../services/api';
import { toast } from 'react-toastify';
import { Pill, Plus, AlertTriangle, TrendingUp, ClipboardList, CheckCircle, Package, Trash2,
Edit, Clock } from 'lucide-react';

const PharmacistDashboard = () => {
  const [medicines, setMedicines] = useState([]);
  const [prescriptions, setPrescriptions] = useState([]);
  const [activeTab, setActiveTab] = useState('prescriptions');
  const [showStockModal, setShowStockModal] = useState(false);
  const [isEditing, setIsEditing] = useState(false);
  const [formData, setFormData] = useState({ id: "", name: "", stock: "", price: "", expiry_date: "" });

  useEffect(() => {
    fetchMedicines();

```

```
fetchPrescriptions();
```

```
}, []);
```

```
const fetchMedicines = async () => {
```

```
  try {
```

```
    const res = await api.get('/pharmacy/medicines');
```

```
    setMedicines(res.data.medicines);
```

```
  } catch (err) { toast.error('Failed to load inventory'); }
```

```
};
```

```
const fetchPrescriptions = async () => {
```

```
  try {
```

```
    const res = await api.get('/prescriptions');
```

```
    setPrescriptions(res.data.prescriptions);
```

```
  } catch (err) { toast.error('Failed to load prescriptions'); }
```

```
};
```

```
const handleSubmit = async (e) => {
```

```
  e.preventDefault();
```

```
  try {
```

```
    if (isEditing) {
```

```
      await api.put(`/pharmacy/medicines/${formData.id}`, formData);
```

```
      toast.success('Medicine updated');
```

```
    } else {
```

```
      await api.post('/pharmacy/medicines', formData);
```

```
      toast.success('Medicine added to inventory');
```

```
    }
```

```
    setShowStockModal(false);
```

```
    setFormData({ id: "", name: "", stock: "", price: "", expiry_date: "" });
```

```
    fetchMedicines();  
  } catch (err) { toast.error('Operation failed'); }
```

```
};
```

```
const deleteMedicine = async (id) => {  
  if (window.confirm('Are you sure you want to delete this medicine?')) {  
    try {  
      await api.delete(`/pharmacy/medicines/${id}`);  
      toast.success('Medicine deleted');  
      fetchMedicines();  
    } catch (err) { toast.error('Delete failed'); }  
  }  
};
```

```
const dispensePrescription = async (id) => {  
  try {  
    await api.patch(`/prescriptions/${id}/dispense`);  
    toast.success('Medicine dispensed correctly');  
    fetchPrescriptions();  
    fetchMedicines();  
  } catch (err) {  
    toast.error(err.response?.data?.message || 'Dispense failed');  
  }  
};
```

```

const openEditModal = (m) => {
  setFormData({
    id: m.id,
    name: m.name,
    stock: m.stock,
    price: m.price,
    expiry_date: m.expiry_date.split("T")[0]
  });

  setIsEditing(true);
  setShowStockModal(true);
};

return (
  <div className="space-y-10 animate-in">
    <header className="flex flex-col md:flex-row justify-between items-start md:items-center gap-8 bg-white p-10 rounded-[2.5rem] border border-slate-200/60 shadow-xl shadow-slate-200/10">
      <div className="flex items-center gap-6">
        <div className="w-20 h-20 bg-blue-600 text-white rounded-3xl flex items-center justify-center shadow-2xl shadow-blue-500/40">
          <Pill size={40} strokeWidth={1.5} />
        </div>
        <div>
          <h1 className="text-4xl font-black text-slate-900 tracking-tight">Pharmacy
Portal</h1>
          <p className="text-slate-500 mt-1 font-semibold flex items-center gap-2">
            <span className="w-2 h-2 bg-blue-500 rounded-full animate-pulse"></span>
            Medical Inventory Management System
          </p>
        </div>
      </div>
    </header>
  </div>

```



```

</div>
<div className="flex flex-wrap gap-4">
  <button
    onClick={() => setActiveTab(activeTab === 'inventory' ? 'prescriptions' :
'inventory')}>
    className="btn-secondary flex items-center gap-2 px-8"
  >
    {activeTab === 'inventory' ? <ClipboardList size={18} /> : <Package size={18}
/>}

    {activeTab === 'inventory' ? 'Pending Orders' : 'Check Inventory'}
  </button>
  {activeTab === 'inventory' && (
    <button onClick={() => { setIsEditing(false); setFormData({ id: ", name: ", stock:
", price: ", expiry_date: " }); setShowStockModal(true); }} className="btn-primary group">
      <Plus size={20} className="transition-transform group-hover:rotate-90" />
      Add Formula
    </button>
  )}
</div>
</header>

```

```

{activeTab === 'prescriptions' ? (
  <div className="card overflow-hidden">
    <div className="p-6 px-10 border-b border-slate-100 bg-slate-50/30 flex justify-
between items-center">
      <h3 className="text-lg font-black text-slate-900 flex items-center gap-3">
        <ClipboardList size={22} className="text-blue-600" />
        Pending Prescriptions queue
      </h3>
      <div className="flex gap-2">
        <span className="badge bg-green-100 text-green-700">Verified</span>

```

</div>

</div>

<div className="overflow-x-auto">

<table className="w-full text-left">

<thead>

<tr className="bg-white border-b border-slate-100">

<th className="px-10 py-5 text-[11px] font-black text-slate-400 uppercase tracking-[0.2em]">Patient & Doctor</th>

<th className="px-10 py-5 text-[11px] font-black text-slate-400

uppercase tracking-[0.2em]">Medical Asset</th>

<th className="px-10 py-5 text-[11px] font-black text-slate-400

uppercase tracking-[0.2em]">Clinical Guidance</th>

<th className="px-10 py-5 text-[11px] font-black text-slate-400

uppercase tracking-[0.2em] text-right">Fulfillment</th>

</tr>

</thead>

<tbody className="divide-y divide-slate-50">

{prescriptions.map(p => (

<tr key={p.id} className="group hover:bg-blue-50/40 transition-all">

<td className="px-10 py-6">

<div>

<p className="font-extrabold text-slate-900">{p.patient_name}</p>

<p className="text-xs font-bold text-blue-500 uppercase tracking-tighter">Dr. {p.doctor_name}</p>

</div>

</td>

<td className="px-10 py-6">

<div className="flex items-center gap-3">

```

        <div className="w-10 h-10 bg-white border border-slate-100
text-blue-600 rounded-xl flex items-center justify-center font-black">
            <Pill size={16} />
        </div>
        <div>
            <p className="font-extrabold text-slate-
900">{p.medicine_name}</p>
            <p className="text-[10px] font-bold text-slate-400
uppercase">Dosage: {p.dosage}</p>
        </div>
    </div>

</td>

<td className="px-10 py-6 max-w-xs">
    <p className="text-xs font-medium text-slate-600 italic leading-
relaxed">

        {p.instructions || 'Standard administration required'}
    </p>
</td>

<td className="px-10 py-6 text-right">
    {p.status === 'pending' ? (
        <button
            onClick={() => dispensePrescription(p.id)}
            className="btn-primary flex items-center gap-2 py-2 px-6 text-
xs ml-auto shadow-sm"
            >
                <CheckCircle size={14} /> Dispense
            </button>
        ) : (
            <span className="badge bg-slate-100 text-slate-

```

500">Fulfilled

)}

</td>

</tr>

))}

</tbody>

</table>

</div>

</div>

): null}

{activeTab === 'inventory' && (

<div className="grid grid-cols-1 lg:grid-cols-4 gap-8">

<div className="card lg:col-span-3 overflow-hidden">

<div className="p-6 px-10 border-b border-slate-100 bg-slate-50/30 flex justify-between items-center">

<h3 className="text-lg font-black text-slate-900 flex items-center gap-3">

<Package size={22} className="text-blue-600" />

Inventory Ledger

</h3>

<button className="text-xs font-bold text-blue-600 hover:underline">Export

CSV</button>

</div>

<div className="overflow-x-auto">

<table className="w-full text-left">

<thead>

<tr className="bg-white border-b border-slate-100">

<th className="px-10 py-5 text-[11px] font-black text-slate-400 uppercase tracking-[0.2em]">Formula</th>

<th className="px-10 py-5 text-[11px] font-black text-slate-400 uppercase tracking-[0.2em]">Availability</th>

```

        <th className="px-10 py-5 text-[11px] font-black text-slate-400
uppercase tracking-[0.2em]">Unit Price</th>
        <th className="px-10 py-5 text-[11px] font-black text-slate-400
uppercase tracking-[0.2em]">Stability</th>
        <th className="px-10 py-5 text-[11px] font-black text-slate-400
uppercase tracking-[0.2em] text-right">Options</th>
    </tr>
</thead>
<tbody className="divide-y divide-slate-50">
    {medicines.map(m => (
        <tr key={m.id} className="group hover:bg-slate-50 transition-all">
            <td className="px-10 py-6">
                <div className="flex items-center gap-4">
                    <div className="w-10 h-10 bg-blue-50 text-blue-600 rounded-
xl flex items-center justify-center font-black">

<Pill size={16} />

                    </div>
                    <span className="font-extrabold text-slate-
900">{m.name}</span>
                </div>
            </td>
            <td className="px-10 py-6">
                <div className="flex flex-col gap-1">
                    <span className={`text-sm font-black ${m.stock < 10 ? 'text-
red-500' : 'text-slate-700'}`}>{m.stock} Units</span>
                    <div className="w-24 h-1.5 bg-slate-100 rounded-full
overflow-hidden">

                        <div className={`${h-full rounded-full transition-all
${m.stock < 10 ? 'bg-red-500 w-1/4' : 'bg-blue-500 w-3/4'}`}></div>

```

```

        </div>
      </div>
    </td>
    <td className="px-10 py-6">
      <span className="text-sm font-black text-slate-
900">${m.price}</span>
    </td>
    <td className="px-10 py-6">
      <div className="flex items-center gap-2 text-[11px] font-bold
text-slate-400">
        <Clock size={12} />
        {new Date(m.expiry_date).toLocaleDateString(undefined, {
month: 'short', year: 'numeric' })}
      </div>
    </td>
    <td className="px-10 py-6 text-right">

    <div className="flex justify-end gap-3">
      <button onClick={() => openEditModal(m)} className="w-8
h-8 flex items-center justify-center text-slate-400 hover:text-blue-600 hover:bg-blue-50 rounded-
lg transition-all">
        <Edit size={16} />
      </button>
      <button onClick={() => deleteMedicine(m.id)} className="w-
8 h-8 flex items-center justify-center text-slate-400 hover:text-red-600 hover:bg-red-50 rounded-
lg transition-all">
        <Trash2 size={16} />
      </button>
    </div>
  </td>
</tr>

```

```

    )))
  </tbody>
</table>
</div>
</div>

```

```

<div className="space-y-8">
  <div className="card bg-blue-600 text-white border-none shadow-xl shadow-
blue-200/50 p-8 overflow-hidden relative group">
    <TrendingUp className="absolute -right-4 -bottom-4 w-32 h-32 text-blue-
500/20 transform -rotate-12 group-hover:scale-110 transition-transform" />
    <p className="text-[11px] font-black text-blue-200 uppercase tracking-widest
mb-4">Inventory Reach</p>
    <p className="text-5xl font-black mb-2">{medicines.length}</p>
    <p className="text-xs font-bold text-blue-100">Validated Formulae</p>
  </div>

```

```

<div className="card p-8 bg-orange-50/50 border-orange-100">
  <div className="flex items-center justify-between mb-6">
    <p className="text-[11px] font-black text-orange-600 uppercase tracking-
widest">Priority Stock</p>
    <AlertTriangle size={18} className="text-orange-500" />
  </div>
  <div className="space-y-4">
    {medicines.filter(m => m.stock < 10).map(m => (
      <div key={m.id} className="flex items-center justify-between p-4 bg-
white rounded-2xl border border-orange-100 shadow-sm">
        <p className="text-xs font-black text-slate-800">{m.name}</p>
        <span className="text-[10px] font-black px-2 py-0.5 bg-red-50 text-

```

```

red-600 rounded-lg">{m.stock} Left</span>
    </div>
  )))
  {medicines.filter(m => m.stock < 10).length === 0 && (
    <div className="text-center py-6">
      <CheckCircle className="mx-auto text-green-500 mb-2" size={24} />
      <p className="text-xs font-bold text-slate-500">All levels
optimal</p>
    </div>
  )}
</div>
</div>
</div>
</div>
  )}

  {showStockModal && (
    <div className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items-center
justify-center z-50 p-4">
      <div className="bg-white rounded-2xl w-full max-w-md p-8 shadow-2xl">

        <h2 className="text-xl font-bold text-slate-800 mb-6">{isEditing ? 'Edit Medicine' : 'Inventory
Addition'}</h2>

        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-slate-700 mb-
1">Medicine Name</label>
            <input className="input-field" required value={formData.name}
onChange={e => setFormData({ ...formData, name: e.target.value })} />
          </div>

```



```

<div className="grid grid-cols-2 gap-4">
  <div>
    <label className="block text-sm font-medium text-slate-700 mb-1">Stock</label>
    <input className="input-field" type="number" required
value={formData.stock} onChange={e => setFormData({ ...formData, stock: e.target.value })}
/>
  </div>
  <div>
    <label className="block text-sm font-medium text-slate-700 mb-1">Price ($)</label>
    <input className="input-field" type="number" step="0.01" required
value={formData.price} onChange={e => setFormData({ ...formData, price: e.target.value })}
/>
  </div>
</div>
<div>
  <label className="block text-sm font-medium text-slate-700 mb-1">Expiry
Date</label>
  <input className="input-field" type="date" required
value={formData.expiry_date} onChange={e => setFormData({ ...formData, expiry_date:
e.target.value })} />
</div>

<div className="flex gap-4 pt-4">
  <button type="button" onClick={() => setShowStockModal(false)}
className="flex-1 px-4 py-2 border rounded-lg text-slate-600 hover:bg-slate-50 transition-
colors font-medium">Cancel</button>
  <button type="submit" className="flex-1 btn-primary">{isEditing ? 'Save
Changes' : 'Add Item'}</button>

```

```
        </div>
      </form>
    </div>
  </div>
  )}
</div>
);
};
```

```
export default PharmacistDashboard;
```

ReceptionistDashboard.jsx

```
import { useEffect, useState } from 'react';
import api from '../services/api';
import { toast } from 'react-toastify';
import {
  UserPlus,
  Receipt,
  Search,
  Plus,
  Trash2,
  Download,
  Edit3,
  Phone,
  MapPin,
  Droplet,
  UserCircle2,
  Mail,
  Lock,
  Calendar,
  Building2
```

```
} from 'lucide-react';
```

```
const ReceptionistDashboard = () => {
```

```
  const [patients, setPatients] = useState([]);
```

```
  const [bills, setBills] = useState([]);
```

```
  const [activeTab, setActiveTab] = useState('billing');
```

```
  // Modal states
```

```
  const [showBillModal, setShowBillModal] = useState(false);
```

```
  const [showRegisterModal, setShowRegisterModal] = useState(false);
```

```
  const [showEditModal, setShowEditModal] = useState(false);
```

```
  // Form states
```

```
  const [newBill, setNewBill] = useState({ patient_id: "", items: [{ description: "", amount: "" }] });
```

```
  const [registerForm, setRegisterForm] = useState({ name: "", email: "", password: "", role:  
'patient' });
```

```
  const [editForm, setEditForm] = useState({ id: "", name: "", age: "", gender: "", blood_group: "",  
phone: "", address: "" });
```

```
  useEffect(() => {
```

```
    fetchPatients();
```

```
    fetchBills();
```

```
  }, []);
```

```
  const fetchPatients = async () => {
```

```
    try {
```

```
      const res = await api.get('/patients');
```

```
      setPatients(res.data.patients);
```

```
    } catch (err) { console.error(err); }
```

```
  };
```

```
const fetchBills = async () => {  
  try {  
    const res = await api.get('/billing');  
    setBills(res.data.bills);  
  } catch (err) { console.error(err); }  
};
```

```
const handleRegisterPatient = async (e) => {  
  e.preventDefault();  
  try {  
    await api.post('/auth/register', registerForm);  
    toast.success('Patient registered successfully');  
    setShowRegisterModal(false);  
    setRegisterForm({ name: "", email: "", password: "", role: 'patient' });  
    fetchPatients();  
  } catch (err) {  
    toast.error(err.response?.data?.message || 'Registration failed');  
  }  
};
```

```
const handleUpdatePatient = async (e) => {  
  e.preventDefault();  
  try {  
    await api.put(`/patients/${editForm.id}`, {  
      name: editForm.name,  
      age: editForm.age,  
      gender: editForm.gender,  
      blood_group: editForm.blood_group,  
      phone: editForm.phone,  
      address: editForm.address  
    });  
  }  
};
```

```

        toast.success('Patient details updated');
        setShowEditModal(false);
        fetchPatients();
    } catch (err) {
        toast.error('Update failed');
    }
};

```

```

const openEditModal = (patient) => {
    setEditForm({
        id: patient.id,
        name: patient.name,
        age: patient.age || "",
        gender: patient.gender || "",
        blood_group: patient.blood_group || "",
        phone: patient.phone || "",
        address: patient.address || ""
    });
    setShowEditModal(true);
};

```

```

const handleCreateBill = async (e) => {
    e.preventDefault();
    const total_amount = newBill.items.reduce((sum, item) => sum + parseFloat(item.amount ||
0), 0);
    try {
        await api.post('/billing', { ...newBill, total_amount });
        toast.success('Invoice generated successfully');
        setShowBillModal(false);
        setNewBill({ patient_id: "", items: [{ description: "", amount: "" }] });
        fetchBills();
    }
};

```

```
    } catch (err) { toast.error('Failed to create bill'); }  
  };
```

```
const addItem = () => setNewBill({ ...newBill, items: [...newBill.items, { description: "",  
amount: "" }] });  
const removeItem = (index) => setNewBill({ ...newBill, items: newBill.items.filter((_, i) => i  
!== index) });
```

```
const downloadInvoice = async (id) => {  
  try {  
    const res = await api.get(`/billing/${id}/pdf`, { responseType: 'blob' });  
    const url = window.URL.createObjectURL(new Blob([res.data]));  
    const link = document.createElement('a');  
    link.href = url;  
    link.setAttribute('download', `invoice_${id}.pdf`);  
    document.body.appendChild(link);  
    link.click();  
  } catch (err) { toast.error('PDF generation failed'); }  
};
```

```
return (  
  <div className="space-y-10 animate-in">  
    <header className="flex flex-col md:flex-row justify-between items-start md:items-  
center gap-6 bg-white p-8 rounded-[2.5rem] border border-slate-200/50 shadow-sm border-b-4  
border-b-blue-600/10">  
      <div>  
        <h1 className="text-4xl font-extrabold text-slate-900 tracking-tight flex items-  
center gap-3">  
          <Building2 className="text-blue-600" size={36} />  
          Reception Desk  
        </h1>  
      </div>  
    </header>  
  </div>
```

```

<p className="text-slate-500 mt-2 font-medium flex items-center gap-2">
  <span className="w-2 h-2 bg-green-500 rounded-full animate-pulse"></span>
  Live system status: All medical departments online
</p>
</div>
<div className="flex flex-wrap gap-4">
  <button
    onClick={() => setActiveTab(activeTab === 'billing' ? 'patients' : 'billing')}
    className="btn-secondary flex items-center gap-2 px-8"
  >
    {activeTab === 'billing' ? <UserCircle2 size={18} /> : <Receipt size={18} />}
    View {activeTab === 'billing' ? 'Patient Registry' : 'Billing Ledger'}
  </button>
  {activeTab === 'billing' ? (
    <button onClick={() => setShowBillModal(true)} className="btn-primary
group">
      <Plus size={18} className="transition-transform group-hover:rotate-90" />
      Generate New Invoice
    </button>
  ) : (
    <button onClick={() => setShowRegisterModal(true)} className="btn-primary
group">
      <UserPlus size={18} className="transition-transform group-hover:scale-110"
/>
      Enroll New Patient
    </button>
  )}
</div>
</header>

```

```

{activeTab === 'billing' ? (
  <div className="card overflow-hidden">
    <div className="p-1 px-4 border-b border-slate-100 bg-slate-50/50 flex justify-
between items-center">
      <span className="text-[10px] font-bold text-slate-400 uppercase tracking-
widest">Financial Records</span>
      <div className="flex gap-2 p-2">
        <div className="w-2 h-2 rounded-full bg-slate-200"></div>
        <div className="w-2 h-2 rounded-full bg-slate-200"></div>
        <div className="w-2 h-2 rounded-full bg-slate-200"></div>
      </div>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-left border-collapse">
        <thead>
          <tr className="bg-white">
            <th className="p-6 text-[11px] font-bold text-slate-400 uppercase
tracking-widest border-b border-slate-100">Series No.</th>
            <th className="p-6 text-[11px] font-bold text-slate-400 uppercase
tracking-widest border-b border-slate-100">Medical Beneficiary</th>
            <th className="p-6 text-[11px] font-bold text-slate-400 uppercase
tracking-widest border-b border-slate-100 text-center">Revenue</th>
            <th className="p-6 text-[11px] font-bold text-slate-400 uppercase
tracking-widest border-b border-slate-100">Service Date</th>
            <th className="p-6 text-[11px] font-bold text-slate-400 uppercase
tracking-widest border-b border-slate-100 text-right">Documents</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-50 group">
          {bills.map(bill => (
            <tr key={bill.id} className="hover:bg-blue-50/30 transition-colors">

```



```

<td className="p-6">
    <span className="px-3 py-1 bg-slate-100 text-slate-600 rounded-lg
font-mono text-xs font-bold border border-slate-200/50">
        #INV-{{bill.id.toString().padStart(4, '0')}}
    </span>
</td>
<td className="p-6">
    <div className="flex items-center gap-4">
        <div className="w-10 h-10 rounded-xl bg-blue-100 flex items-
center justify-center text-blue-700 font-bold">
            {{bill.patient_name[0]}}
        </div>
        <div>
            <p className="font-bold text-slate-
800">{{bill.patient_name}}</p>
            <p className="text-xs text-slate-500">{{bill.patient_email}}</p>
        </div>
    </div>
</td>
<td className="p-6 text-center">
    <span className="text-lg font-black text-slate-
900">${bill.total_amount.toLocaleString()}</span>
</td>
<td className="p-6">
    <div className="flex items-center gap-2 text-slate-600 font-
medium">
        <Calendar size={14} className="text-slate-400" />
        {{new Date(bill.created_at).toLocaleDateString(undefined, {
month: 'short', day: 'numeric', year: 'numeric' })}}
    </div>
</td>

```

```

        <td className="p-6 text-right">
            <button
                onClick={() => downloadInvoice(bill.id)}
                className="inline-flex items-center gap-2 px-4 py-2 bg-blue-50
text-blue-700 rounded-xl font-bold text-xs hover:bg-blue-600 hover:text-white transition-all
transform active:scale-95 shadow-sm shadow-blue-200/50"
            >
                <Download size={14} strokeWidth={3} />
                PDF INVOICE
            </button>
        </td>
    </tr>
    )}
</tbody>
</table>
</div>
{bills.length === 0 && (
    <div className="text-center py-24 bg-white">
        <div className="inline-flex p-6 bg-slate-50 rounded-full mb-4">
            <Receipt size={40} className="text-slate-300" />
        </div>
        <h3 className="text-lg font-bold text-slate-800">No transactions
recorded</h3>
        <p className="text-slate-500 text-sm">Create your first invoice to see the
billing history</p>
    </div>
    )}
</div>
): null}
{activeTab === 'patients' && (

```

```

<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-8">
  {patients.map(p => (
    <div key={p.id} className="card-hover group bg-white rounded-[2rem] border
border-slate-200/60 shadow-lg shadow-slate-200/20 overflow-hidden relative">
      <div className="absolute top-0 right-0 p-4 opacity-0 group-hover:opacity-100
transition-opacity">
        <button
          onClick={() => openEditModal(p)}
          className="w-10 h-10 bg-white shadow-xl border border-slate-100 text-
blue-600 rounded-2xl flex items-center justify-center hover:scale-110 active:scale-95 transition-
all"
        >
          <Edit3 size={18} />
        </button>
      </div>

      <div className="p-8 pb-4">
        <div className="w-16 h-16 bg-gradient-to-br from-blue-500 to-blue-700
text-white rounded-[1.5rem] flex items-center justify-center font-black text-2xl shadow-xl
shadow-blue-200 mb-6">
          {p.name[0]}
        </div>
        <h3 className="font-black text-xl text-slate-900 leading-tight mb-
1">{p.name}</h3>
        <p className="text-xs font-bold text-slate-400 uppercase tracking-widest
flex items-center gap-1.5">
          <Mail size={12} strokeWidth={3} />
          {p.email}
        </p>
      </div>
    )
  )}
</div>

```

```

<div className="p-8 pt-0 grid grid-cols-2 gap-4 mt-4">
  <div className="bg-slate-50 rounded-2xl p-4 text-center border border-
slate-100">
    <p className="text-[10px] font-bold text-slate-400 uppercase mb-
1">Biological Age</p>
    <p className="text-lg font-black text-slate-900">{p.age || '--'} <span
className="text-[10px] text-slate-400 uppercase">Yrs</span></p>
  </div>
  <div className="bg-slate-50 rounded-2xl p-4 text-center border border-
slate-100">
    <p className="text-[10px] font-bold text-slate-400 uppercase mb-
1">Blood Type</p>
    <p className="text-lg font-black text-red-600 flex items-center justify-
center gap-1">
      <Droplet size={14} strokeWidth={4} />
      {p.blood_group || 'N/A'}
    </p>
  </div>
  <div className="col-span-full bg-blue-50/50 rounded-2xl p-4 flex items-
center gap-3 border border-blue-100/50">
    <Phone size={16} className="text-blue-600" />
    <span className="text-sm font-bold text-blue-900">{p.phone || 'Contact
pending'}</span>
  </div>
  <div className="col-span-full flex items-center gap-3 px-1">
    <MapPin size={14} className="text-slate-400 shrink-0" />
    <span className="text-xs font-medium text-slate-500 line-clamp-
1">{p.address || 'Address unverified'}</span>
  </div>
</div>

```

```

</div>
    )))
    {patients.length === 0 && (
      <div className="col-span-full py-24 text-center bg-white rounded-[2.5rem]
border border-dashed border-slate-300">
        <UserCircle2 size={48} className="mx-auto text-slate-200 mb-4" />
        <p className="text-lg font-bold text-slate-700">Database is empty</p>
        <p className="text-slate-400 text-sm">Register your first patient to begin
operations</p>
      </div>
    )}
  </div>
)}

{/* Register Patient Modal */}
{showRegisterModal && (
  <div className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items-center
justify-center z-50 p-4">
    <div className="bg-white rounded-3xl w-full max-w-md p-8 shadow-2xl">
      <h2 className="text-2xl font-bold text-slate-800 mb-6 flex items-center gap-2">
        <UserPlus className="text-blue-700" size={24} /> New Patient Account
      </h2>
      <form onSubmit={handleRegisterPatient} className="space-y-4">
        <div>
          <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Full Name</label>
          <div className="relative">
            <UserCircle2 className="absolute left-3 top-1/2 -translate-y-1/2 text-
slate-400" size={18} />
            <input
              className="input-field pl-10 h-12"

```

```

        placeholder="Enter patient name"
        required
        value={registerForm.name}
        onChange={(e) => setRegisterForm({ ...registerForm, name:
e.target.value })}
    />
</div>
</div>
<div>
    <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Email Address</label>
    <div className="relative">
        <Mail className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400"
size={18} />
        <input
            className="input-field pl-10 h-12"
            type="email"
            placeholder="patient@email.com"
            required
            value={registerForm.email}
            onChange={(e) => setRegisterForm({ ...registerForm, email:
e.target.value })}
        />
    </div>
</div>
</div>
<div>
    <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Temporary Password</label>
    <div className="relative">
        <Lock className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400"
size={18} />

```

```

        <input
          className="input-field pl-10 h-12"
          type="password"
          placeholder="Initial login password"
          required
          value={registerForm.password}
          onChange={(e) => setRegisterForm({ ...registerForm, password:
e.target.value })}
        />
      </div>
    </div>
    <div className="flex gap-4 pt-6">
      <button
        type="button"
        onClick={() => setShowRegisterModal(false)}
        className="flex-1 h-12 border border-slate-200 rounded-xl text-slate-600
hover:bg-slate-50 font-bold transition-all"
      >
        Cancel
      </button>
      <button type="submit" className="flex-1 h-12 btn-primary shadow-lg
shadow-blue-700/20">
        Register
      </button>
    </div>
  </form>
</div>
</div>
)}

```

```

    { /* Edit Patient Modal */ }
    { showEditModal && (
      <div className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items-center
justify-center z-50 p-4">
        <div className="bg-white rounded-3xl w-full max-w-2xl p-8 shadow-2xl">
          <h2 className="text-2xl font-bold text-slate-800 mb-6 flex items-center gap-2">
            <Edit3 className="text-blue-700" size={24} /> Edit Medical Profile:
            {editForm.name}
          </h2>
          <form onSubmit={handleUpdatePatient} className="grid grid-cols-1 md:grid-
cols-2 gap-6">
            <div>
              <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Age</label>
              <input
                className="input-field h-12"
                type="number"
                value={editForm.age}
                onChange={(e) => setEditForm({ ...editForm, age: e.target.value })}
              />
            </div>
            <div>
              <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Gender</label>
              <select
                className="input-field h-12"
                value={editForm.gender}
                onChange={(e) => setEditForm({ ...editForm, gender: e.target.value })}
              >
                <option value="">Select Gender</option>
                <option value="male">Male</option>

```



```

        <option value="female">Female</option>
        <option value="other">Other</option>
    </select>
</div>
<div>
    <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Blood Group</label>
    <select
        className="input-field h-12"
        value={editForm.blood_group}
        onChange={(e) => setEditForm({ ...editForm, blood_group: e.target.value
}}}
    >
        <option value="">Select Blood Group</option>
        <option value="A+">A+</option>
        <option value="A-">A-</option>
        <option value="B+">B+</option>
        <option value="B-">B-</option>
        <option value="O+">O+</option>
        <option value="O-">O-</option>
        <option value="AB+">AB+</option>
        <option value="AB-">AB-</option>
    </select>
</div>
<div>
    <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Phone Number</label>
    <div className="relative">
        <Phone className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-
400" size={18} />
        <input

```

```

        className="input-field pl-10 h-12"
        value={editForm.phone}
        onChange={(e) => setEditForm({ ...editForm, phone: e.target.value })}
      />
    </div>
  </div>
  <div className="col-span-full">
    <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Residential Address</label>
    <div className="relative">
      <MapPin className="absolute left-3 top-4 text-slate-400" size={18} />
      <textarea
        className="input-field pl-10 min-h-[100px] pt-3"
        value={editForm.address}
        onChange={(e) => setEditForm({ ...editForm, address: e.target.value
    }}}

      ></textarea>
    </div>
  </div>
  <div className="col-span-full flex gap-4 pt-6">
    <button
      type="button"
      onClick={() => setShowEditModal(false)}
      className="flex-1 h-12 border border-slate-200 rounded-xl text-slate-600
hover:bg-slate-50 font-bold transition-all"
    >
      Cancel
    </button>
    <button type="submit" className="flex-1 h-12 btn-primary shadow-lg
shadow-blue-700/20">
      Update Profile

```

```

        </button>
      </div>
    </form>
  </div>
</div>
)}

```

```

{ /* Create Bill Modal */ }
{ showBillModal && (
  <div className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items-center
justify-center z-50 p-4">
    <div className="bg-white rounded-3xl w-full max-w-2xl p-8 shadow-2xl max-h-
[90vh] overflow-y-auto">
      <h2 className="text-2xl font-bold text-slate-800 mb-8">Generate Medical
Invoice</h2>
      <form onSubmit={handleCreateBill} className="space-y-6">
        <div>
          <label className="block text-sm font-bold text-slate-700 mb-2 uppercase
tracking-wide">Select Patient</label>
          <select
            className="input-field h-12"
            required
            value={newBill.patient_id}
            onChange={e => setNewBill({ ...newBill, patient_id: e.target.value })}
          >
            <option value="">-- Choose --</option>
            {patients.map(p => <option key={p.id} value={p.id}>{p.name}
({p.email})</option>)}
          </select>
        </div>

```

```

<div className="space-y-4">
  <div className="flex justify-between items-center">
    <label className="text-sm font-bold text-slate-700 uppercase tracking-
wide">Service Items</label>
    <button type="button" onClick={addItem} className="text-blue-700
font-bold text-sm flex items-center gap-1 hover:underline">
      <Plus size={16} /> Add Charge
    </button>
  </div>
  {newBill.items.map((item, idx) => (
    <div key={idx} className="flex gap-4 items-end">
      <div className="flex-1">
        <input
          placeholder="Description (e.g. Consultation)"
          className="input-field"
          required
          value={item.description}
          onChange={e => {
            const items = [...newBill.items];
            items[idx].description = e.target.value;
            setNewBill({ ...newBill, items });
          }}
        />
      </div>
      <div className="w-32">
        <input
          placeholder="Amount"
          type="number"
          className="input-field"
          required
          value={item.amount}
          onChange={e => {

```

```

        const items = [...newBill.items];
        items[idx].amount = e.target.value;
        setNewBill({ ...newBill, items });
      }}
    />
  </div>
  {newBill.items.length > 1 && (
    <button type="button" onClick={() => removeItem(idx)}
className="p-2 text-red-500 hover:bg-red-50 rounded-lg mb-1">
      <Trash2 size={20} />
    </button>
  )}
</div>
)}}
</div>

<div className="pt-6 border-t flex items-center justify-between">
  <div>
    <p className="text-slate-500 text-sm">Total Invoice Amount</p>
    <p className="text-3xl font-bold text-slate-800">
      ${newBill.items.reduce((sum, item) => sum + parseFloat(item.amount ||
0), 0).toFixed(2)}
    </p>
  </div>
  <div className="flex gap-4">
    <button type="button" onClick={() => setShowBillModal(false)}
className="px-6 py-2 border rounded-xl font-bold text-slate-600">Cancel</button>
    <button type="submit" className="btn-primary px-8">Issue
Invoice</button>
  </div>
</div>
</form>

```

```

        </div>
      </div>
    )}
  </div>
);
};

```

```
export default ReceptionistDashboard;
```

Register.jsx

```

import { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import api from '../services/api';
import { toast } from 'react-toastify';
import { Building2, User, Mail, Lock, Send } from 'lucide-react';

const Register = () => {
  const [formData, setFormData] = useState({ name: '', email: '', password: '', role: 'patient' });
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    try {
      await api.post('/auth/register', formData);
      toast.success('Account established. You can now authenticate.');
```

```

        setLoading(false);
    }
};

return (
    <div className="min-h-screen flex items-center justify-center bg-[#f8fadc] p-6 animate-in
relative overflow-hidden">
        { /* Ambient Background Elements */ }
        <div className="absolute top-[-10%] right-[-10%] w-[40%] h-[40%] bg-blue-400/10
blur-[120px] rounded-full"></div>
        <div className="absolute bottom-[-10%] left-[-10%] w-[40%] h-[40%] bg-indigo-
400/10 blur-[120px] rounded-full"></div>

        <div className="max-w-xl w-full relative z-10">
            <div className="bg-white rounded-[3rem] shadow-2xl p-12 md:p-16 border border-
slate-200/60 relative overflow-hidden">
                <div className="absolute top-0 left-0 w-full h-2 bg-gradient-to-r from-blue-600 to-
indigo-600"></div>

                <div className="text-center mb-10">
                    <Link to="/" className="inline-flex items-center justify-center w-24 h-24 bg-
gradient-to-br from-indigo-600 to-blue-700 text-white rounded-[2.5rem] mb-8 shadow-2xl
shadow-indigo-500/40 transform hover:scale-110 transition-transform duration-500">
                        <User size={48} strokeWidth={1} />
                    </Link>
                    <h1 className="text-5xl font-black text-slate-900 tracking-tight mb-
4">Health<span className="text-blue-600">Quest</span></h1>
                    <div className="flex items-center justify-center gap-2">
                        <div className="h-px w-8 bg-slate-200"></div>
                        <p className="text-slate-400 font-black uppercase tracking-[0.3em] text-
[10px]">Patient Enrollment Portal</p>
                        <div className="h-px w-8 bg-slate-200"></div>

```

</div>

</div>

<form onSubmit={handleSubmit} className="space-y-6">

<div className="space-y-2">

<label className="text-[10px] font-black text-slate-400 uppercase tracking-widest ml-2 px-1">Legal Name</label>

<div className="relative group">

<User className="absolute left-5 top-1/2 -translate-y-1/2 text-slate-400 group-focus-within:text-blue-600 transition-colors" size={20} />

<input

className="w-full bg-slate-50 border border-slate-200 h-16 rounded-2xl pl-14 pr-6 text-slate-900 font-bold focus:bg-white focus:ring-4 focus:ring-blue-500/10 focus:border-blue-500 transition-all outline-none text-lg"

placeholder="Enter full name"

required

value={formData.name}

onChange={(e) => setFormData({ ...formData, name: e.target.value })}

/>

</div>

</div>

<div className="space-y-2">

<label className="text-[10px] font-black text-slate-400 uppercase tracking-widest ml-2 px-1">Contact Email</label>

<div className="relative group">

<Mail className="absolute left-5 top-1/2 -translate-y-1/2 text-slate-400 group-focus-within:text-blue-600 transition-colors" size={20} />

<input

className="w-full bg-slate-50 border border-slate-200 h-16 rounded-2xl pl-14 pr-6 text-slate-900 font-bold focus:bg-white focus:ring-4 focus:ring-blue-500/10 focus:border-blue-500 transition-all outline-none text-lg"


```

        type="email"
        placeholder="patient@medical.com"
        required
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
      />
    </div>
  </div>
  <div className="space-y-2">
    <label className="text-[10px] font-black text-slate-400 uppercase tracking-
widest ml-2 px-1">Passphrase</label>
    <div className="relative group">
      <Lock className="absolute left-5 top-1/2 -translate-y-1/2 text-slate-400
group-focus-within:text-blue-600 transition-colors" size={20} />
      <input
        className="w-full bg-slate-50 border border-slate-200 h-16 rounded-2xl
pl-14 pr-6 text-slate-900 font-bold focus:bg-white focus:ring-4 focus:ring-blue-500/10
focus:border-blue-500 transition-all outline-none text-lg"
        type="password"
        placeholder="••••••••"
        required
        value={formData.password}
        onChange={(e) => setFormData({ ...formData, password: e.target.value
    }}}
  />
</div>
</div>
  <button
    type="submit"
    disabled={loading}

```

```
        className="w-full bg-slate-900 hover:bg-black text-white h-16 rounded-2xl
text-xl font-black tracking-tight flex items-center justify-center gap-3 mt-10 shadow-2xl shadow-
slate-200 transition-all active:scale-[0.98] group"
```

```
>
```

```
{loading ? (
```

```
    <div className="h-6 w-6 border-4 border-white/30 border-t-white rounded-
full animate-spin"></div>
```

```
): (
```

```
<>
```

```
    Establish Records
```

```
    <Send size={20} className="group-hover:translate-x-1 group-
hover:translate-y-[-1px] transition-transform" />
```

```
</>
```

```
)}  
</button>
```

```
</form>
```

```
<div className="mt-12 text-center pt-8 border-t border-slate-100">
```

```
  <p className="text-slate-500 font-semibold text-sm">
```

```
    Already enrolled? <Link to="/login" className="text-blue-600 font-black
hover:text-blue-700 ml-1 underline decoration-2 underline-offset-4">Authorize Now</Link>
```

```
</p>
```

```
</div>
```

```
</div>
```

```
<div className="mt-8 flex justify-center gap-8">
```

```
  <p className="text-[10px] font-black text-slate-400 uppercase tracking-[0.2em]
flex items-center gap-2">
```

```
    <span className="w-1.5 h-1.5 bg-green-500 rounded-full"></span>
```

```
    Encrypted Storage
```

```
</p>
```

```
</div>
```

```

        </div>
    </div>
);
};

export default Register;

```

Staff.jsx

```

import { useEffect, useState } from 'react';
import api from '../services/api';
import { toast } from 'react-toastify';
import { UserPlus, Trash2, ShieldCheck, Mail, UserCircle2, Briefcase } from 'lucide-react';

const Staff = () => {
    const [staff, setStaff] = useState([]);
    const [showModal, setShowModal] = useState(false);
    const [formData, setFormData] = useState({ name: "", email: "", password: "", role: 'receptionist'
});

    useEffect(() => {
        fetchStaff();
    }, []);

    const fetchStaff = async () => {
        try {
            const res = await api.get('/users');
            // Filter out admin and patient roles to show only support staff and doctors
            const supportRoles = ['receptionist', 'pharmacist', 'lab'];
            const filteredStaff = res.data.users.filter(u => supportRoles.includes(u.role));
            setStaff(filteredStaff);

```

```
    } catch (err) {  
      toast.error('Failed to load staff members');  
    }  
  };  
};
```

```
const handleSubmit = async (e) => {  
  e.preventDefault();  
  try {  
    await api.post('/auth/register', formData);  
    toast.success(`${formData.role} created successfully`);  
    setShowModal(false);  
    setFormData({ name: "", email: "", password: "", role: 'receptionist' });  
    fetchStaff();  
  } catch (err) {  
    toast.error(err.response?.data?.message || 'Failed to create staff account');  
  }  
};
```

```
const deleteStaff = async (id) => {  
  if (window.confirm('Are you sure you want to remove this staff member?')) {  
    try {  
      await api.delete(`/users/${id}`);  
      toast.success('Staff member removed');  
      fetchStaff();  
    } catch (err) {  
      toast.error('Failed to delete staff member');  
    }  
  }  
};
```

```
const roleColors = {
```

```
receptionist: 'bg-emerald-100 text-emerald-700',
pharmacist: 'bg-indigo-100 text-indigo-700',
lab: 'bg-rose-100 text-rose-700'
};
```

```
return (
  <div className="space-y-10 animate-in">
    <header className="flex flex-col md:flex-row justify-between items-start md:items-center gap-8 bg-white p-10 rounded-[2.5rem] border border-slate-200/60 shadow-xl shadow-slate-200/10">
      <div className="flex items-center gap-6">
        <div className="w-20 h-20 bg-blue-600 text-white rounded-3xl flex items-center justify-center shadow-2xl shadow-blue-500/40">
          <Briefcase size={40} strokeWidth={1.5} />
        </div>
        <div>
          <h1 className="text-4xl font-black text-slate-900 tracking-tight">Staff
Operations</h1>
          <p className="text-slate-500 mt-1 font-semibold flex items-center gap-2">
            <span className="w-2 h-2 bg-blue-500 rounded-full animate-pulse"></span>
            Workforce Management & Credentials
          </p>
        </div>
      </div>
      <button
        onClick={() => setShowModal(true)}
        className="btn-primary group px-8"
      >
        <UserPlus size={20} className="transition-transform group-hover:scale-110" />
        Onboard New Staff
      </button>
```

</header>

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-8">
  {staff.map((member) => (
    <div key={member.id} className="card-hover group bg-white rounded-[2rem]
border border-slate-200/60 shadow-lg shadow-slate-200/20 overflow-hidden relative p-8">
      <div className="absolute top-0 right-0 p-4 opacity-0 group-hover:opacity-100
transition-opacity">
        <button
          onClick={() => deleteStaff(member.id)}
          className="w-10 h-10 bg-white shadow-xl border border-slate-100 text-
red-600 rounded-2xl flex items-center justify-center hover:scale-110 active:scale-95 transition-
all"
        >
          <Trash2 size={18} />
        </button>
      </div>

      <div className="flex flex-col items-center text-center mb-8">
        <div className="w-20 h-20 bg-gradient-to-br from-slate-100 to-slate-200 text-
slate-500 rounded-[1.5rem] flex items-center justify-center font-black text-3xl shadow-inner mb-
6 ring-4 ring-slate-50">
          {member.name[0]}
        </div>
        <h3 className="font-black text-xl text-slate-900 leading-tight mb-2
uppercase">{member.name}</h3>
        <span className={`${px-4 py-1 rounded-xl text-[10px] uppercase font-black
tracking-[0.1em] shadow-sm ${roleColors[member.role]}`}>
          {member.role === 'lab' ? 'Lab Technician' : member.role}
        </span>
      </div>
    </div>
  )}
```

```

    <div className="space-y-4 pt-6 border-t border-slate-50">
      <div className="flex items-center gap-3 bg-slate-50 p-4 rounded-2xl border
border-slate-100">
        <Mail size={16} className="text-slate-400 shrink-0" />
        <span className="text-sm font-bold text-slate-600
truncate">{member.email}</span>
      </div>
      <div className="flex items-center justify-center gap-2 text-[10px] font-black
text-emerald-600 uppercase tracking-widest px-1">
        <ShieldCheck size={14} />
        Verified Personnel
      </div>
    </div>
  </div>
)}
{staff.length === 0 && (
  <div className="col-span-full card border-dashed py-12 text-center text-slate-400
italic">
    No support staff accounts found. Use the button above to add members.
  </div>
)}
</div>

```

```

{showModal && (
  <div className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items-center
justify-center z-50 p-4">
    <div className="bg-white rounded-3xl w-full max-w-md p-8 shadow-2xl
relative">
      <div className="mb-8">
        <h2 className="text-2xl font-bold text-slate-800">Add Staff Member</h2>
        <p className="text-slate-500 text-sm">Create credentials for medical support

```

teams</p>

</div>

<form onSubmit={handleSubmit} className="space-y-5">

<div>

<label className="block text-xs font-bold text-slate-400 uppercase tracking-widest mb-2 px-1">Full Name</label>

<div className="relative">

<UserCircle2 className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400" size={18} />

<input

className="input-field pl-10 h-12"

placeholder="Enter name"

required

value={formData.name}

onChange={(e) => setFormData({ ...formData, name: e.target.value })}

/>

</div>

</div>

<div>

<label className="block text-xs font-bold text-slate-400 uppercase tracking-widest mb-2 px-1">Email Address</label>

<div className="relative">

<Mail className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400" size={18} />

<input

className="input-field pl-10 h-12"

type="email"

placeholder="email@hospital.com"

required


```

        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
      />
    </div>
  </div>

```

```

<div>
  <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Access Role</label>
  <div className="relative">
    <Briefcase className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-
400" size={18} />
    <select
      className="input-field pl-10 h-12"
      required
      value={formData.role}
      onChange={(e) => setFormData({ ...formData, role: e.target.value })}
    >
      <option value="receptionist">Receptionist</option>
      <option value="pharmacist">Pharmacist</option>
      <option value="lab">Lab Technician</option>
    </select>
  </div>
</div>

```

```

<div>
  <label className="block text-xs font-bold text-slate-400 uppercase
tracking-widest mb-2 px-1">Initial Password</label>
  <input
    className="input-field h-12"
    type="password"
    placeholder="Minimum 6 characters"
  />

```

```

        required
        minLength={6}
        value={formData.password}
        onChange={(e) => setFormData({ ...formData, password: e.target.value
    }}}

    />
</div>

<div className="flex gap-4 pt-6">
    <button
        type="button"
        onClick={() => setShowModal(false)}
        className="flex-1 h-12 border border-slate-200 rounded-xl text-slate-600
hover:bg-slate-50 font-bold transition-all"
    >
        Cancel
    </button>
    <button type="submit" className="flex-1 h-12 btn-primary shadow-lg
shadow-blue-700/20">
        Create Account
    </button>
</div>
</form>
</div>
</div>
    )}
</div>
);
};

export default Staff;

```

REFERENCES

1. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
This book provides a comprehensive understanding of software development methodologies, system design, and implementation techniques used in building large-scale systems like hospital management systems.
2. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.
A detailed reference on system development life cycle, requirements engineering, and system architecture relevant to healthcare software systems.
3. Welling, L., & Thomson, L. (2017). *PHP and MySQL Web Development*. Addison-Wesley.
Explains backend development and database integration concepts useful for building dynamic web-based hospital systems.
4. Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6), 80–83.
Describes the use of Node.js for scalable backend systems, applicable to hospital management platforms.
5. MySQL Development Team. (2024). *MySQL Documentation*.
Available at: <https://dev.mysql.com/doc/>
Official documentation for MySQL database used for managing hospital data such as patient records and billing.
6. React Development Team. (2024). *React Official Documentation*.

Available at: <https://react.dev>

Provides guidance on building dynamic and responsive user interfaces for web-based systems.

7. Express.js Development Team. (2024). *Express.js Documentation*.

Available at: <https://expressjs.com>

Documentation for building backend APIs and handling server-side logic in web applications.

8. Chart.js Development Team. (2024). *Chart.js Documentation*.

Available at: <https://www.chartjs.org>

Explains how to create interactive charts and dashboards for data visualization in hospital systems.

9. Mozilla Developer Network (MDN). (2024). *Web Development Documentation*.

Available at: <https://developer.mozilla.org>

Provides resources for HTML, CSS, and JavaScript used in frontend development.

10. GitHub, Inc. (2024). *GitHub Documentation*.

Available at: <https://docs.github.com>

Documentation for version control and collaboration tools used in software development.